

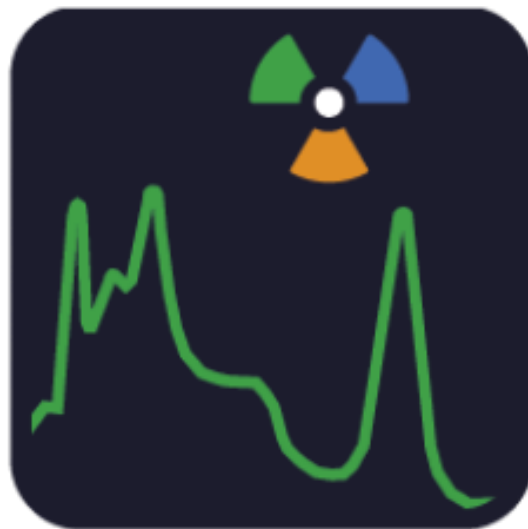
Medical Radiation Physics,
Lund University,
Sweden

October 7, 2026
LuRaCs version: 0.5.1

LuRaCs - Reference Manual

author

Erik Ewald



LUND
UNIVERSITY



Contents

1	Getting Started	3
1.1	Introduction	3
1.1.1	LuRaCs-H3	3
1.2	Installation	3
1.2.1	With <code>pip</code>	4
1.2.2	From source	4
1.3	Connecting Devices	5
1.4	Compatible file formats	5
1.5	Package dependencies and Resources	6
1.6	Terms & Object Definitions	6
1.6.1	Abbreviations	8
2	Interface	9
2.1	Top Tabs	9
2.1.1	Spectrum	9
2.1.2	Spectrogram	12
2.1.3	Map	15
2.2	Bottom Tabs	16
2.2.1	Spectrum Info	16
2.2.2	Spectrum ROIs	18
2.2.3	Spectrogram ROIs	18
2.2.4	Real Time Values	18
2.2.5	Detectors	19
2.2.6	Isotopics	21
2.2.7	Console	22
2.2.8	System Log	23
2.3	Gamma Tools	23
2.3.1	Calibration	23
2.3.2	Deconvolution	23
2.3.3	Efficiency	24
2.3.4	Resolution	25
2.4	Command Line Arguments	25
2.5	Command Line Interface (CLI)	26
2.6	Detector Connection & Control	29
2.7	Data Store	29
2.8	Settings	34
3	Examples	37
3.1	Quick Starting Several Radiacode Detectors for mobile measurements	37
3.2	Connecting Detective X using direct ethernet	37
3.3	Preparing and Running the LU mobile detector setup	38
4	Internal Function	39
4.1	Code Structure	39
4.2	Wrapping Detectors	41
4.3	Algorithms	42
4.3.1	Peak Fitting	42
4.3.2	Peak Search	42
4.3.3	Deconvolution	42

4.4	File Types	42
4.4.1	Spectrum, ROI and Instrument (.xml)	42
4.4.2	Spectrogram (.db)	47
4.4.3	Detector Response Matrix (.npz)	48
4.4.4	Simple map tracks (.geojson)	48
4.5	Embedded map	49

1 Getting Started

Licence, Warranty and Research Use

This software is free and open-source software distributed under the terms of the GNU General Public License, version 3 (GPLv3).

The software is provided “AS IS” and WITHOUT WARRANTY OF ANY KIND, to the maximum extent permitted by applicable law. The warranty disclaimer and limitation of liability contained in Sections 15 and 16 of the GPLv3 apply to the software.

Lund University and its contributors make no representation or warranty that the software is accurate, reliable, error-free, uninterrupted, or suitable for any particular purpose.

Research Use

This software is intended as a research and experimental tool for gamma spectroscopy and mobile radiation measurements. It is not intended to constitute a certified measurement system, and measurement results should not be assumed to be accurate or suitable for regulatory, safety-critical, medical, environmental, or other consequential applications without appropriate independent validation.

Measurement results may be affected by the detector, electronics, calibration, measurement geometry, environmental conditions, acquisition parameters, data processing, and other factors.

Users are responsible for evaluating and validating the software and its results for their particular research application.

To the maximum extent permitted by applicable law, Lund University and its contributors accept no liability for any loss, damage, decision, action, or omission arising from or in connection with the use of the software or reliance on measurements or results obtained using the software.

Nothing in this notice excludes or limits any liability that cannot legally be excluded or limited under applicable law.

The complete GNU General Public License version 3 is provided with the software.

1.1 Introduction

1.1.1 LuRaCs-H3

LuRaCs-H3 is a light weight version of the compiled program indented for system with limited storage. It excludes heavy components to reduce file size of the executable. It is approximately 250 MB smaller than the regular version. The following systems are excluded in the H3 version.

- The embedded map, position data can still be recorded.

1.2 Installation

LuRaCs distributed as binary does not require any installs before use, with the exception of usb drivers on Windows.

Before installing, ensure that Python version 3.10, 3.11, 3.12 or 3.13 is installed on the system. The Python installation can be verified by opening *cmd* on windows or a Terminal

on MacOS and Linux and entering `python --version` or `python3 --version`, if Python is correctly installed the version will be displayed.

1.2.1 With pip

When Python has been correctly set up, make a new folder for the install and open *cmd* or a terminal in that folder. On Windows enter the following commands in *cmd*:

```
# Make a new python environment
python -m venv .venv

# Activate the environment
.\.venv\Scripts\activate

# Install
pip install luracs

# Run
LuRaCs
```

On Linux and MacOS run the following bash commands:

```
# Make a new python environment
python3 -m venv .venv

# Activate the environment
source .venv/bin/activate

# Install
pip install luracs

# Run
LuRaCs
```

This will install all dependencies for the full version of the application and provide the start script LuRaCs.

1.2.2 From source

LuRaCs can be installed from source by cloning the GitHub repository. Installing from source requires a C/C++-compiler for compiling *Cython* extensions. When installing from source to edit the code, run `pip install -e .[dev]` to install in editable mode. This will also install development tools like `pyinstaller` and `pytest`.

```
# Clone the the GitHub repo
git clone --recursive https://github.com/EBELU/LuRaCs.git
cd LuRaCs

# Install with pip
pip install .
```

```
# Run
LuRaCs
```

1.3 Connecting Devices

Table 1.1: Currently implemented devices

Model	Manufacturer	Connections	Note
RadiaCode-1xx	Radiacode	USB, BLE	
Raysid	Raysid	BLE	
digiBase	Orectec Ametek	USB	Requires Ortec Firmware
digiDart	Orectec Ametek	USB	
DetectiveX	Orectec Ametek	NETWORK	

In order to use USB connected devices on Linux the udev-rules must be updated by adding the following permissions depending on the device.

```
# Radiacode
SUBSYSTEM=="usb", ATTR{idVendor}=="0483", ATTR{idProduct}=="f123", TAG+="uaccess",
↳ MODE="0660"

# digiBase/digiDart
SUBSYSTEM=="usb", ATTRS{idVendor}=="0a2d", GROUP="users", MODE="0666"
```

Connecting devices over the network likely requires setup specific to that instrument so no general guide is provided. See example 3.2 for how this can be done for the Ortec Ametek Detective X.

Connecting a digiBase requires the files containing the necessary firmware. These files can be found in the install directories of the *Maestro* and *Gamma Vision* software provided by Ortec. LuRaCs does not provide these files and users are responsible for acquiring these files in accordance with applicable licences. Once the files have been identified they can be reached by LuRaCs by saving them in the driver library. Select **File -> Import Drivers** and select the files. If they appear in **File -> Drivers** they can now be reached by LuRaCs.

1.4 Compatible file formats

LuRaCs is compatible with a selection of file formats, both standard and more niche third party formats. However, LuRaCs is not primarily intended to support general spectrum analysis so if you are looking for a general analysis application consider using InterSpec¹ instead.

¹*Sandialabs/InterSpec*. Sandia National Laboratories, Aug. 28, 2025. URL: <https://github.com/sandialabs/InterSpec> (visited on 08/29/2025).

Table 1.2: Compatible spectrum formats.

Source	Extension	Contained Data
Radiacode	.xml	Counts, background, live time, real time, calibration
<i>Several</i>	.n42	Counts, background, live time, real time, calibration
Ortec	.spe	Counts, live time, real time, calibration
Canberra	.tka	Counts, live time, real time

Table 1.3: Compatible track formats.

Source	Extension
Radiacode	.rctrk

1.5 Package dependencies and Resources

1.6 Terms & Object Definitions

Spectrum is a data object that contains count histograms from a radiation detector and associated meta data. A spectrum contains the following information:

- Spectrum name.
- Number of channels, fore- and background must have the same number of channels.
- Optional remark or description.
- Detector identifier.
- Detector model.
- Fore- and Background spectrum.
- ROIs and analysis results from evaluating the ROI on the spectrum like counts and parameters from a curve fit.
- Calibration coefficients for converting channel number to energy.
- Calibrated X-axis expressed in keV.
- Reference to the instrument used for calculations.

Region Of Interest (ROI) is a spectrum region bounded by energy. They can be applied both in a spectrum and in a spectrogram. When applied to a spectrum a Gaussian function can be fitted to a peak contained inside the ROI.

When applied to spectrograms ROIs do not perform any curve fitting, only the summed counts inside the ROI are calculated. The time series produced from a spectrogram ROI can be viewed separately in the map. Spectrograms can use special ROIs called *Calculation ROIs* or CalcROIs. These are calculated from data produced by two other ROIs by combining their values with addition, subtraction or division. For example, CAL_0 (a Calculation ROI) can show the results of ROI_0 / ROI_1 where ROI_0 is placed over the K-40 peak and ROI_1 is placed over the Cs-137 peak.

Spectrogram is the primary method for recording time series data. It allows for measuring the difference in counts in each channel between time points separated by a fixed time. It also interfaces with the GPS function and can store geolocational information for mobile measurements. A spectrogram is a rather complicated object and is discussed further in other parts of this manual, see section 4.4.2.

Options when creating a spectrogram are

- Detector to use for data.
- Spectrogram name.
- Save interval, the time between each row.
- If channels should be concatenated, concatenating by a factor 2 means summing two channels to one, halving to total number of channels. Useful for reducing filesize and improving visual analysis.

If spectrum corresponding to the detector used for the spectrogram is energy calibrated this calibration is also used for the spectrogram.

Instrument s are divided into *Generic* and *Unique* instruments. *Generic Instrument* represent an instrument model. All instruments of the same model share some properties, of which the *Generic Instrument* stores the following:

- Model name.
- Manufacturer.
- Detector material.
- Sensitive volume size.
- Energy Resolution.
- Intrinsic Energy Efficiency.

A *Unique Instrument* represents a specific instrument of a certain model. The *Generic Instruments* serve as a basis for quickly making a new *Unique Instrument*. A *Unique Instrument* store the same information as a *Generic Instrument* in addition to an energy calibration. *Unique Instruments* can be assigned to a spectrum from which certain tools can pull required data for calculations and they can store the energy calibration for instruments that do not store it themselves.

Detector denotes an external instrument for detecting radiation that LuRaCs can sample data from. They can be connected through USB, Bluetooth or a network. Each type of detector have their own 'client'. LuRaCs interacts with detectors through clients which contain the code needed to interact with the physical detector but each client can be implemented differently. This structure makes detector implementations in LuRaCs flexible and easily extensible.

1.6.1 Abbreviations

Table 1.4: List of abbreviations used in this manual

CLI	Command Line Interface
CPS	Counts Per Second
GUI	Graphical User Interface
ML-EM	Maximum Likelihood - Expectation Maximisation

2 Interface

2.1 Top Tabs

2.1.1 Spectrum

The spectrum view shows spectra as a count histograms, retrieved from a connected device or loaded from a file. It can show several spectra combined in one plot or separately. This can be changed under **Options** -> **Tabbed-/Combined Spectrum View**.

Background spectra can be viewed as one the three following methods *No background*, *Background overlay* and *background subtract*. *Background overlay* shows the background spectrum overlaid on top of the main spectrum. *Background subtract* does channel wise subtraction of the background from the foreground spectrum, this is a hard background subtraction and will effect ROI calculations on the spectrum. Both *Background overlay* and *background subtract* only show values of count rate.

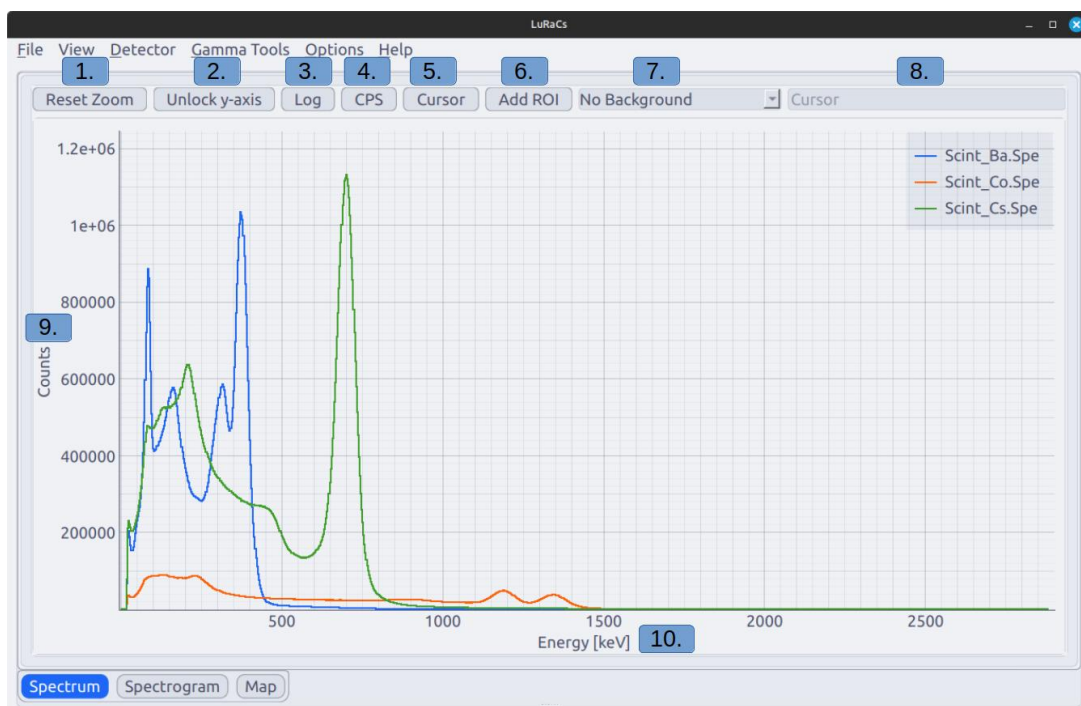


Figure 2.1: Spectra of Ba-133, C-60 and Cs-137 displayed in *Combined Spectrum View*.

1. **Rest Zoom** resets the zoom of the spectrum and locks the y-axis if unlocked.
2. **Unlock y-axis** allows the y-axis to be zoomed independently and disables autoscaling.
3. **Log** set the scale of the y-axis to logarithmic.
4. **CPS** normalises the spectrum to the live time, or real time if live time is not available but real time is.
5. **Cursor** toggles the spectrum cursor, see Figure 2.3.
6. **Add ROI** adds a customisable *region of interest* to the spectrum, see Figure 2.2.
7. Switch the use of background spectrum. Options are *No background*, *Background overlay* and *background subtract*. See the section above.
8. Display for info from the cursor, see Figure 2.3.
9. The y-axis show either total counts or time normalised counts in units of 1/s.
10. The x-axis always display energy in units of keV.

Regions of interest (ROI) are sub regions of the spectrum bounded in energy and are

useful in the analysis of spectrum peaks. Spectrum peaks can in most cases be approximated with a Gaussian function and knowing the function parameters to describe is useful for many analyses. LuRaCs provides a method for fitting Gaussian functions to peaks inside ROIs and do so by default. These fits are required by other tools in the application. However, ROIs can also be configured to not perform fits, in these cases only the total ROI counts are calculated.

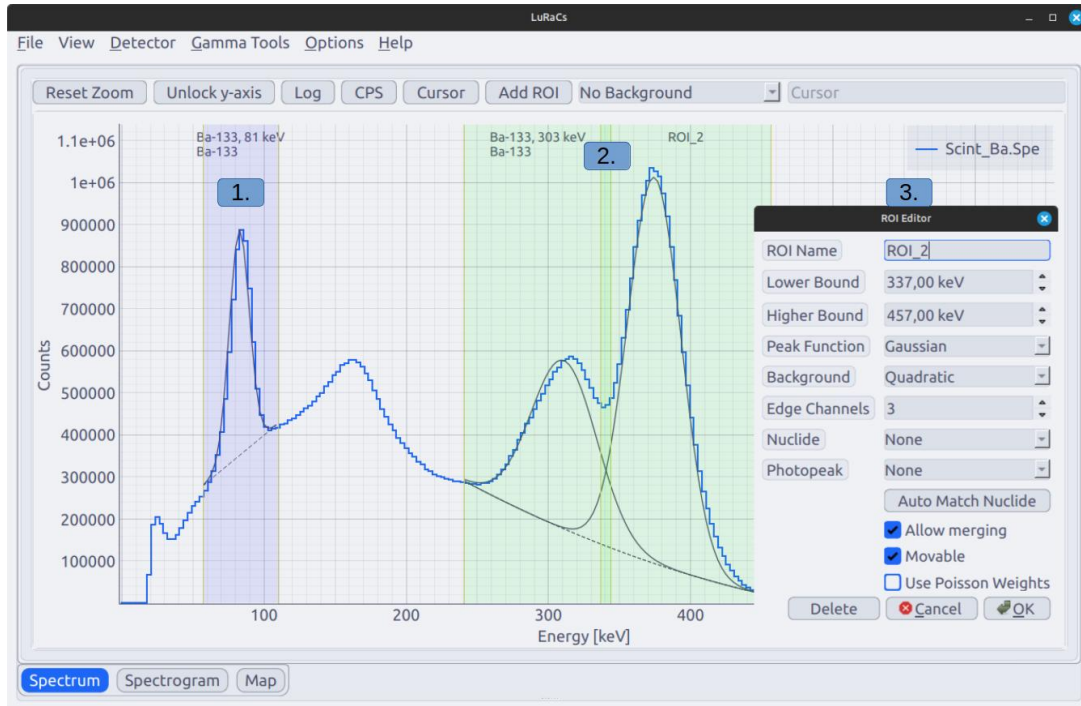


Figure 2.2: A Ba-133 spectrum with three ROIs, two are merged and fitted to the same background. The ROI editor is open for the right most ROI.

1. A ROI with one peak and a fitted Gaussian peak and linear background.
2. Two merged ROIs that fit two Gaussian peaks on a shared quadratic background.
3. The ROI editor for 'ROI_2', see Table 2.1.

Table 2.1: ROI Editor options

Field	Description
ROI Name	An alias for the ROI used for display purposes and easier identification.
Lower Bound	The lower energy bound of the ROI.
Upper Bound	The upper energy bound of the ROI.
Peak Function	The type function fitted to the peak. Options are: <i>Gaussian</i> , <i>None</i>
Background	Type of function used as peak background. Options are: <i>None</i> , <i>Linear</i> , <i>Quadratic</i>
Edge Channels	The number of channels inside of ROI from the upper and lower bounds of the ROI used to fit the background.
Nuclide	Assign a nuclide from the reference library to the ROI. Used for calculations by other tools.
Photopeak	Choose the specific photon emission associated with the peak in the ROI. Options are based on the chosen nuclide.
Auto Match Nuclide	Attempt to automatically find the Nuclide and Emission of the peak. If any nuclides are checked and shown in the spectrum the match is limited to these. Blind matches are often unreliable.
Allow Merging	Allow the ROI to merge with other ROIs with merging allowed.
Movable	If the ROI can be moved by using the mouse.
Use Poission Weights	Apply a $1/\sigma$ weight to the optimisation to force the peak to emulate a Poission distribution.

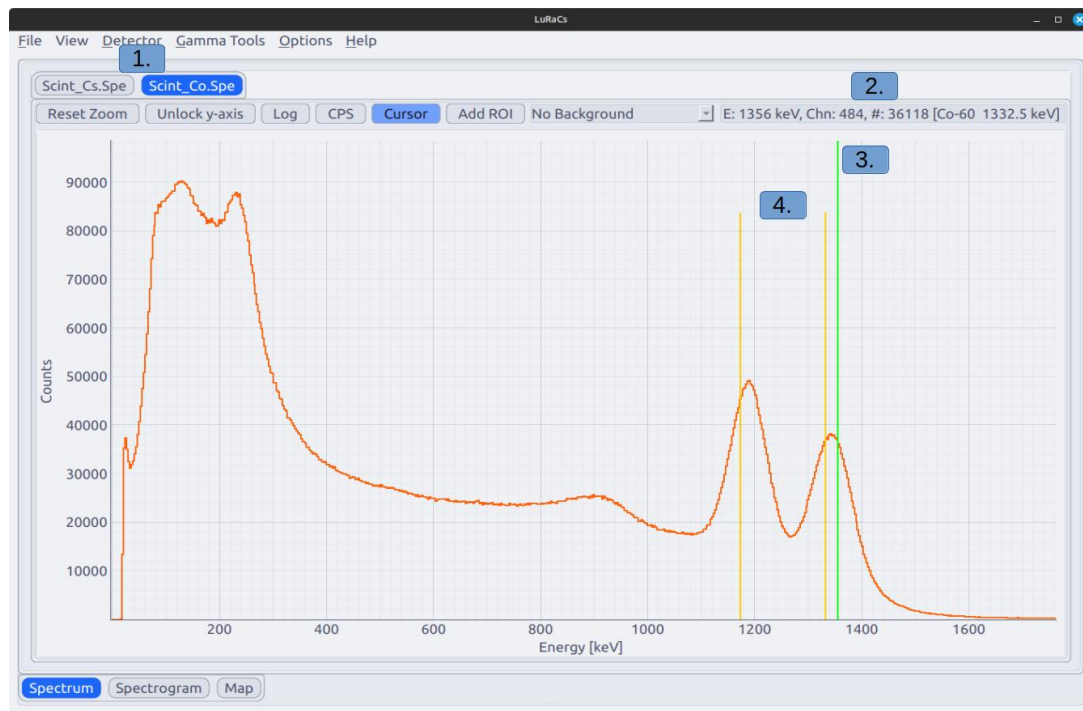


Figure 2.3: Co-60 spectrum with an active cursor.

1. Tabs for selecting spectrum in *Tabbed Spectrum View*.
2. Information about the cursor, it shows position of the cursor in units of energy, corresponding channels number, the number of counts in the channel, closest reference emission line as [*nuclide Emission Energy Decay chain if applicable*]. Channel and channel counts are only displayed in tabbed spectrum view.
3. Cursor in the spectrum.
4. Emission lines of the suggested nuclide. These are always to closest to the cursor and does not weigh intensity or anything else. Can be disabled by **View -> Spectrum -> Show Cursor Emissions**.

2.1.2 Spectrogram

A spectrogram allows time-series measurements to be conducted. A spectrogram stores data at specified time intervals and can be displayed in a waterfall plot. It is also the method LuRaCs uses for mobile measurements. If a GPS is connected the geolocation data provided is automatically stored in the spectrogram. These measurements can be visualised on the embedded Map.

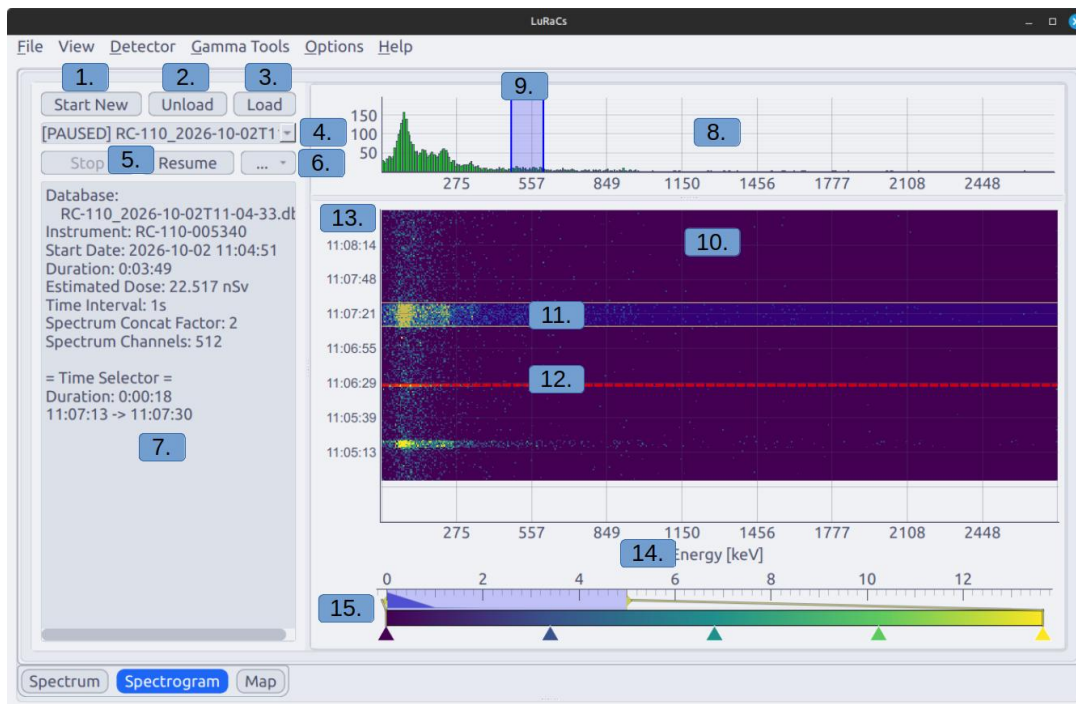


Figure 2.4: Measured spectrogram with the *Time Selector* active and one ROI.

1. Opens a dialog for starting a new spectrogram.
2. Close the viewed spectrogram.
3. Shortcut to the Data Store Spectrogram page, see section 2.7.
4. Selector for currently loaded spectrograms. Status is shown to the left as [ACTIVE], [LOADED] or [PAUSED].
5. Pause or Resume data collections, requires that the same instrument used to collect the previous data to be connected.
6. Options menu, see Figure 2.5
7. Current information stored in the spectrogram and information about the *Time Selector* if active.
8. Count histogram. Shows either total accumulated spectrum upon loading a measured spectrogram, summed channel counts inside the *Time Selector* if active or counts in the latest row.
9. ROI illustrated in the histogram, can be deleted with Mouse-2.
10. Waterfall diagram of the spectrogram.
11. *Time Selector* to study a specific time interval. Can be toggled in the options menu (6.).
12. Break line, illustrates that the time between measured points exceeds the Time Interval. Can indicate a connection and caution should be taken to verify data integrity. Can be toggled in the options menu (6.).
13. Y-axis displays the timestamps of when each row was measured.
14. X-axis display spectrum energy if calibrated.
15. Color slider for the waterfall diagram. Color map can be changed by clicking Mouse-2 on the gradient bar.

The spectrogram format is based around an SQLite database, see section 4.4.2 for further details on the format. The following data is stored for each time point in the spectrogram:

- Timestamp.
- Average count rate measured since the last time point.

- Average dose rate measured since the last time point.
- Device temperature at the time the data is written.
- Longitude at the time the data is written, if a GPS is available.
- Latitude at the time the data is written, if a GPS is available.
- Spectrum difference since the last written data point, stored as compressed binary.
- Meta data, stored as a utf-8 encoded and zlib compressed json-string. Examples of stored metadata include extended GPS data.

Exporting a spectrogram is done through the data store, quickly accessed via the *Load*-button (2.).

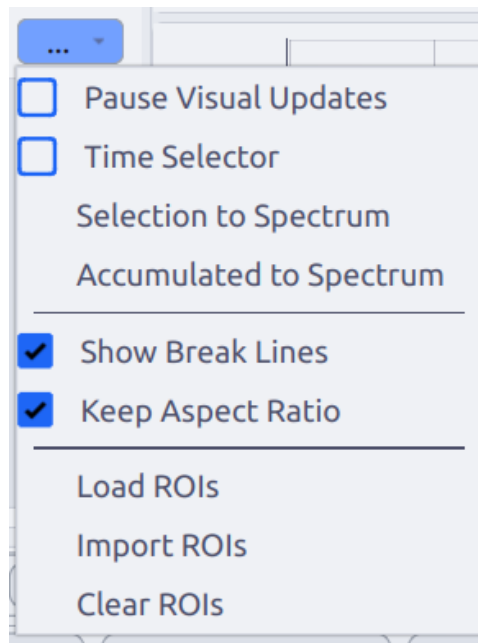


Figure 2.5: Dropdown options menu in spectrogram pane, see Table 2.2.

Table 2.2: Description of fields in the spectrogram options menu

Field	Description
Pause Visual Updates	Stop the view spectrogram from updating while the measurement is still going on in the background. This does not stop the recording of the spectrogram.
Time Selector	Show a selection ROI in the spectrogram (11.) that can select specific time interval and show the sum of the channels in the top count histogram (8.).
Live Time	Live time of the foreground
ROIs	How many ROIs are saved with the spectrum, if any.
Selection to Spectrum	Export the selected interval to the datastore as a spectrum.
Accumulated to Spectrum	Export all accumulated counts to the datastore as a spectrum.
Show Break Lines	If the red lines (12.) indicating if the interval between two rows exceed the specified save interval should be shown.
Keep Aspect Ratio	If the pixels in the spectrogram should always be square. For detectors with many channels it can be useful to turn this off. If this is off the pixels in the spectrogram may not align with the corresponding bar in the count histogram.
Load ROIs	Shortcut the to ROI tab of the Data Store.
Import ROIs	Load ROIs from a file.
Clear ROIs	Remove all spectrogram ROIs

2.1.3 Map

Loading a map enables viewing of geographic measurements made with a spectrogram and a GPS. Maps can be loaded from online API requiring an internet connection or loaded from a *.pmtiles*-file for offline use. Compatible APIs and files are offered by several vendors, the software *QGis* offers support for exporting *.pmtiles* from other formats using plugins.

Disclaimer LuRaCs does not provide or distribute map data. Users are responsible for ensuring that any map data or tile services they use with LuRaCs are obtained and used in accordance with the applicable licenses, terms, and usage restrictions.

The map is rendered using Qt's web engine making it a powerful but heavy tool. Using offline maps can use a lot of system memory since the server needs to cache tiles for a smooth view. Use a system with enough memory for this method. However, both the map and datapoints are rendered using GPU acceleration with WebGL allowing for 10 000s of points to be displayed without excessive stress on the system. For reference, a Raspberry Pi 5 with 8 GB of RAM can render a 1.4 GB map file without any stutter, total memory usage was approximately 2 GB.

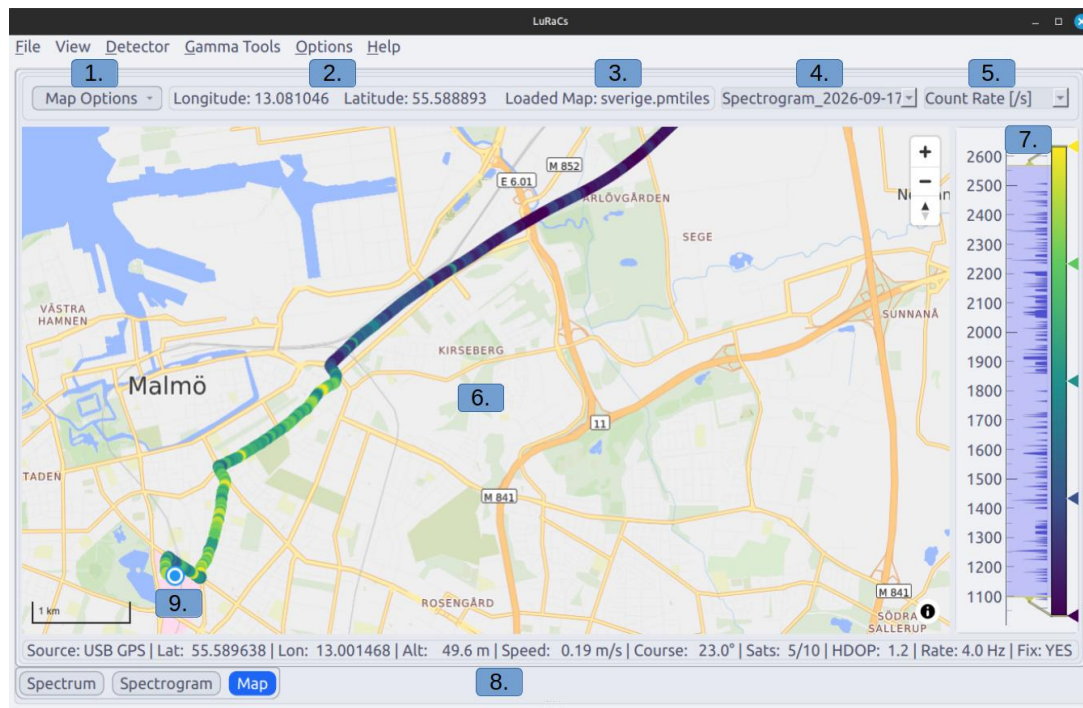


Figure 2.6: Map with a loaded track and connected GPS.

1. Options menu for loading maps and importing and exporting simple tracks.
2. Longitude and Latitude of the cursor.
3. Name of the loaded map.
4. List of loaded spectrograms, if the selected contains no geolocation data no points are displayed.
5. Selector for what data is shown. total Count Rate, Dose Rate or count rate inside a loaded ROI.
6. Main map view.
7. Color gradient selector for the track points and a histogram over measured values. Color map can be changed by clicking Mouse-2 on the gradient bar.
8. Live data from the GPS.
9. Current GPS position, always displayed as a white and blue circle.

Load a map is done from Map Options -> Load Online/Offline Map. An offline map takes the form of a *.pmltiles* files and choosing this options prompts the selection of such a file. Choosing to load an online map prompts the entering of a URL to a tile serving API, possibly including an API key. The checkbox allows for saving the last used URL. If a personal API key is used be aware that URLs are saved unencrypted.

2.2 Bottom Tabs

2.2.1 Spectrum Info

Displays information about loaded spectra, both from files and detectors.

	Spectrum	Type	Counts	Live Time	Real Time	Calibrated	Instrument
1	RC-103-011143	Foreground	74 793	2:23:19	None	True	RC-103-011143
2	RC-103G-001618	Foreground	3 106 563	3 days, 14:29:42	None	True	None
3	RC-110-005340	Foreground	166 808	2:23:01	None	True	RC-110-005340
4	digiBase-88491	Foreground	315 789	0:12:01	0:12:02	False	None
5	DetectiveX_192.168.50.2	Foreground	178 536	0:11:53	0:11:59	True	None
6	2025-07-25-1033_fontan.Spe	Foreground	582 238	0:10:01	0:10:03	True	None
7	HPGeEU152Corrected.TKA	Foreground	887 018	0:10:00	0:10:23	False	None

Figure 2.7: Spectrum Info Tab. For imported spectra with other formats than XML or N42 file extensions are kept in the name.

Table 2.3: Description of the columns in the Spectrum Info tab

Column	Description
Column 0	Color button allowing for changing the color of a spectrum. Options button marked with '...', see Table 2.4.
Spectrum	The name of the spectrum.
Type	<i>Foreground</i> or <i>Background</i> , these share the same name in the 'Spectrum'-column for the same spectrum.
Live Time	The live time of the measurement, accounting for dead time.
Real Time	The real time of the measurement.
Calibrated	<i>True</i> or <i>False</i> , if an energy calibration is detected and applied.
Instrument	Which instrument is assigned to the spectrum, see section 1.6

Table 2.4: Description of the options menu opened by the button marked '...' on each row. Backgrounds are limited to only the 'Remove' option.

Field	Description
Save	Save the spectrum to the Data Store.
Export	Export the spectrum to a stand alone file of the following formats: XML, CSV, xlsx .
Hide/Show	Toggle visibility of the spectrum in the spectrum view. ROIs are not applied to hidden spectra.
Edit	Edit properties of the spectrum, see ??.
Remove Spectrum	Remove the spectrum and disconnect the detector connected to the spectrum.

2.2.2 Spectrum ROIs



Figure 2.8: Spectrum ROIs tab

1. Clear all loaded ROIs from all spectra.
2. Debug view of all data contained in the loaded ROIs.
3. Apply time normalisation to the calculated ROI data.
4. Export the calculated data and the live of each ROI to a CSV-file.
5. Save ROI references to the Data Store, see section 2.7, for later access. These save the settings of the ROI but not the data.
6. Icon to display if the spectrum is background subtracted since this effects the results of ROI calculations.

2.2.3 Spectrogram ROIs

Spectrogram ROIs differ from Spectrum ROIs in that do no curve fitting. When applied to a spectrogram they sum the counts from all the channels within their bounds and do so for every row in the spectrogram. The results are displayed in count rate, the number of counts are normalised to the save interval, for easier comparison if the recorded spectrograms use different save intervals.

For use in mobile measurements where several ROIs is monitored at the same time a 'Compact Mode' can be used. This is enabled under **View -> Spectrogram ROIs -> Compact Mode**. This strips everything except the y-ticks and the plot itself.

Calculation ROIs are plots calculated from the results of other ROIs. **These perform element wise operations and do not account for differences in measurement parameters.** Element 0 from ROI 0 is combined with Element 0 of ROI 1, regardless of measurement date or spectrogram save interval.

2.2.4 Real Time Values

Plots the values sampled for a detector in real time. Not all detectors provide both Count Rate and Dose Rate, in these cases the detector is still registered to the plot but nothing is shown. Y-axes can be made logarithmic by right clicking the axis **Plot Options -> Transforms -> Log Y**.

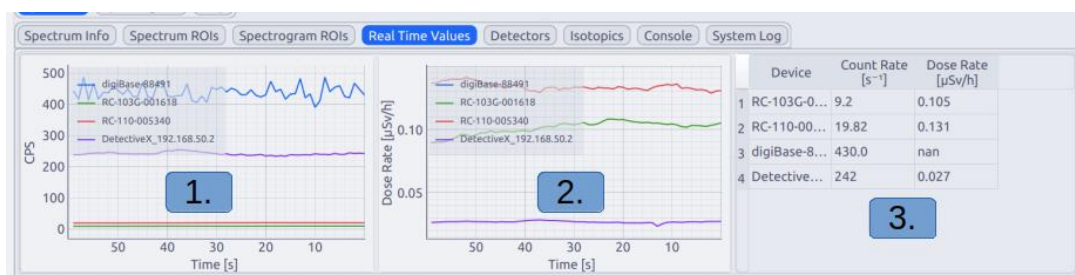


Figure 2.9: Real Time Values with four different detectors connected.

1. Count Rate plot.
2. Dose Rate plot, y-axis is always μSv.
3. Latest values sampled from the corresponding detector.

2.2.5 Detectors

Information and status of connected detectors are displayed in this tab.

	Detector	Temperature	Battery	Charging	Voltage	Status	Type	Connection
1	DetectiveX_192.168.50.2	-155°C	100%	False	nan	CONNECTED	detective_x	NETWORK
2	digiBase-88491	nan	nan	False	750.0V	CONNECTED	digibase	USB
3	RC-103-011143	None	None	None	None	CONNECTION_FAILED	radiacode	USB
4	RC-103G-001618	22.1°C	98.29%	False	nan	CONNECTED	radiacode	BLE
5	RC-110-005340	22.0°C	97.97%	False	nan	ERROR	radiacode	BLE

Figure 2.10: List of connected detectors. An entry can not be removed, if a detector is disconnected it is listed as STOPPED.

Table 2.5: Description of the columns in the Detectors tab

Column	Description
Column 0	Options button marked with '...', see Table 2.6.
Detector	The name of the detector.
Temperature	The temperature reading reported by the detector in °C.
Battery	Reported battery charge in %.
Charging	If the detector reports that it is charging.
Voltage	The actual voltage reported by the detector in V.
Status	Detector state machine reporting the current status of the detector. • UNINITIALIZED - Detector client created but not started. • CONNECTING - The detector is starting. • CONNECTED - The detector is connected and running. Default for normal detector function. • CONNECTION_FAILED - Detector start failed. Check System Log. • CONNECTION_LOST - Connection was lost after a successful start. Not universally implemented. • STOPPING - The detector has received and acknowledged the command to stop. • STOPPED - The detector has successfully stopped. • ERROR - An error occurred while connected and the detector is no longer functioning as expected.
Type	Type of detector by the name defined in the detector client.
Connection	How the detector is connected. Can be BLE, USB or NETWORK.

Table 2.6: Description of the options menu in the detector tab, opened with the button marked '...' in the table.

Field	Description
Disconnect	Sends a stop signal to the detector.
Reset Spectrum	Signals to the selected detector to clear its spectrum.
Start Acquisition	Signals to the selected detector to start spectrum acquisition.
Stop Acquisition	Signals to the selected detector to stop spectrum acquisition.
Make Instrument	Shortcut for creating a <i>Unique Instrument</i> for the detector based on a Generic Instrument and save the current calibration of the connected spectrum. Useful for detectors that do not store their own energy calibration.
Settings	Opens the detector settings dialog, see Figure 2.11 and Figure 2.12.

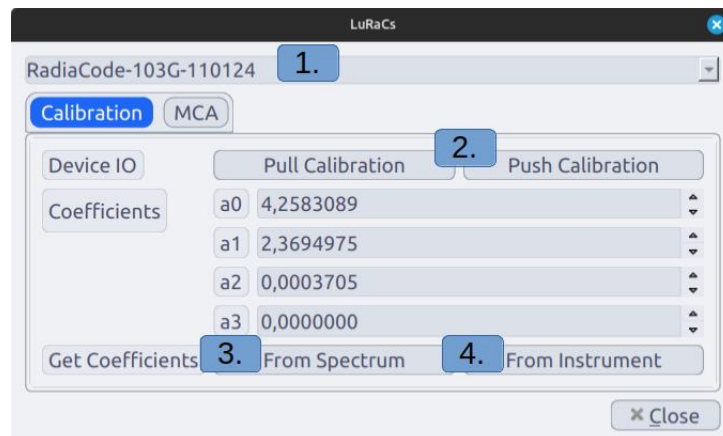


Figure 2.11: Detector settings dialog showing the options for setting device calibration.

1. Unique Instrument from the library to pull settings from.
2. Interact with the detector. **Pull Calibration** will request a calibration from the detector set in the coefficients buffer, not possible for all detectors. **Push Calibration** will call the setter function for calibration, not possible for all detectors.
3. Pull the calibration coefficients from the spectrum connected to the detector and set in the coefficients buffer.
4. Pull the calibration coefficients from the instrument set in (1.) and set in the coefficients buffer

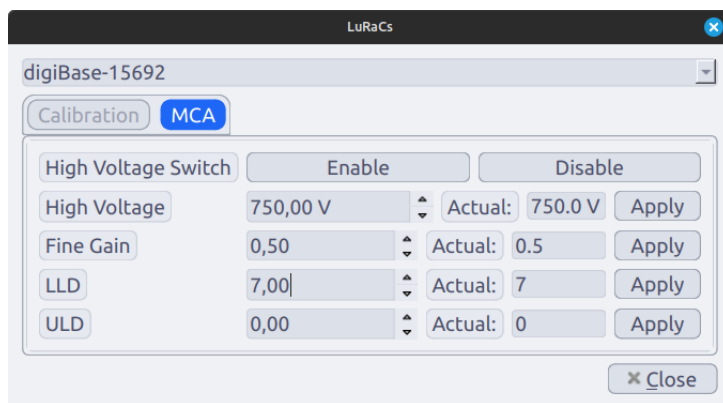


Figure 2.12: Detector settings dialog showing the options for MCA settings. The first column of spinboxes show the requested setting and the second row shows the actual value reported by the instrument. **Apply** signals the entered value to the detector’s setter function.

LLD Lower Level Discriminator
 ULD Upper Level Discriminator

2.2.6 Isotopics

LuRaCs provides a library of data for a selection of common radionuclides and can be viewed in this tab. These can be used to draw reference lines for emissions in the spectrum plot. Most of the nuclide data is sourced from *Laboratoire National Henri Becquerel*, a few nuclides not provided by LNHB had be sourced from elsewhere. A citation is provided for each nuclide and full references can be found under **Help -> Bibliography**.

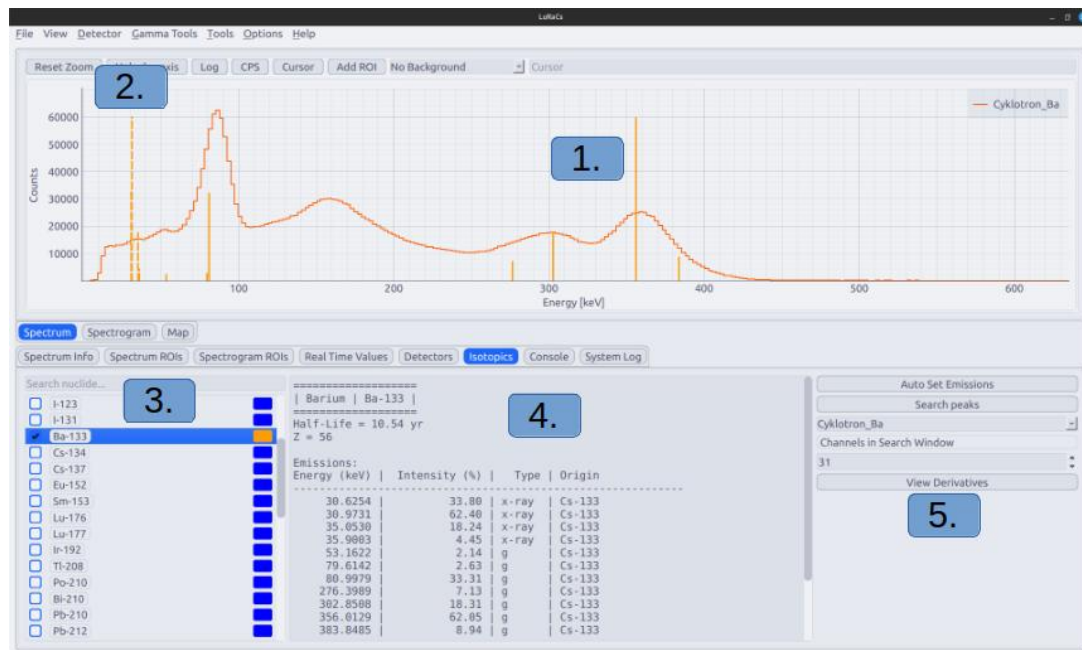


Figure 2.13: Ba-133 spectrum with Ba-133 selected in the Isotopes Tab.

1. Spectrum and gamma emission lines, drawn as solid lines. Line height corresponds to emission intensity,
2. X-ray emission lines, drawn as dashed lines. Line height corresponds to emission intensity.
3. Nuclide list, select nuclide with the checkbox to the left and change colour of the lines with the right coloured button.
4. Information of the current clicked nuclide.
5. Peak automation, see below.

Auto matching nuclides to ROIs is accomplished by using the centroid of a fitted peak and finding emission close in energy. The matching is weighted by the emission intensity so a strong emission will be chosen over a weak of they are the same distance from the peak centroid. By selecting nuclides the auto matching of nuclides to ROIs, including from using the button inside the *ROI Editor*, are limited emission from the selected nuclides. Using auto match nuclides against all possible candidates are often unreliable, especially if the energy calibration is imperfect.

The peak search is based on Savitzky-Golay Smoothing and the filter derivatives to compute peak candidates. These candidates are then filtered and the accepted peaks are added as ROIs. The length of the window in the Savitzky-Golay filter can be set in units of channels.

2.2.7 Console

The console allows for entering text commands to control the application and running scripts for basic automation. See subsection 2.5 for information on available commands and their usage.

The Console can be used for light automation by using the `exec` command and providing a file path. The file is executed line by line and can be used to quickly start set up a measurement session.

2.2.8 System Log

The System Log displays messages from the application and detector clients. If something crashes the error and traceback will be displayed here.

2.3 Gamma Tools

2.3.1 Calibration

Calibration is the operation of determining the energy each channel corresponds to.



Figure 2.14: Calibration window of a spectrum with 4 ROIs and assigned nuclides.

1. Which spectrum should be calibration. Only data from one spectrum can be used for one calibration.
2. Add a calibration point not associated with a ROI. Measured and Reference energies must be set manually.
3. Current calibration points detected and information about them. By unchecking a row it will be ignored in the calculation.
4. Polynomial degree of the calibration function and the current coefficients. Coefficients can be changed manually but will be overwritten by pressing **Calculate**.
5. The spectrum instrument. Can not be changed here, instead edit the spectrum before starting the calibration.
6. Plot showing the energy as a function of channels based on the latest calculated coefficients.
7. Assign the coefficients to the *Unique Instrument* set in (5.).
8. Assign the coefficients and recalculate the x-axis for the spectrum selected in (1.), updating the view in the Spectrum-tab. This will not change the calibration of an already started spectrogram.

2.3.2 Deconvolution

This tool does more than simply reducing the effect of energy resolution but will still be referred to as a deconvolution operation. Deconvolution in this sense is more similar to reconstruction as, like image reconstruction in nuclear medicine, it employs the ML-EM

algorithm. By using a detector response matrix, most simply acquired using Monte Carlo simulations, the effects of the detector resolution, scattering in the detector and other effects can be reduced. The response matrix represents the expected detector response to a photon emitted at a certain energy, see Table 4.10 and section ??

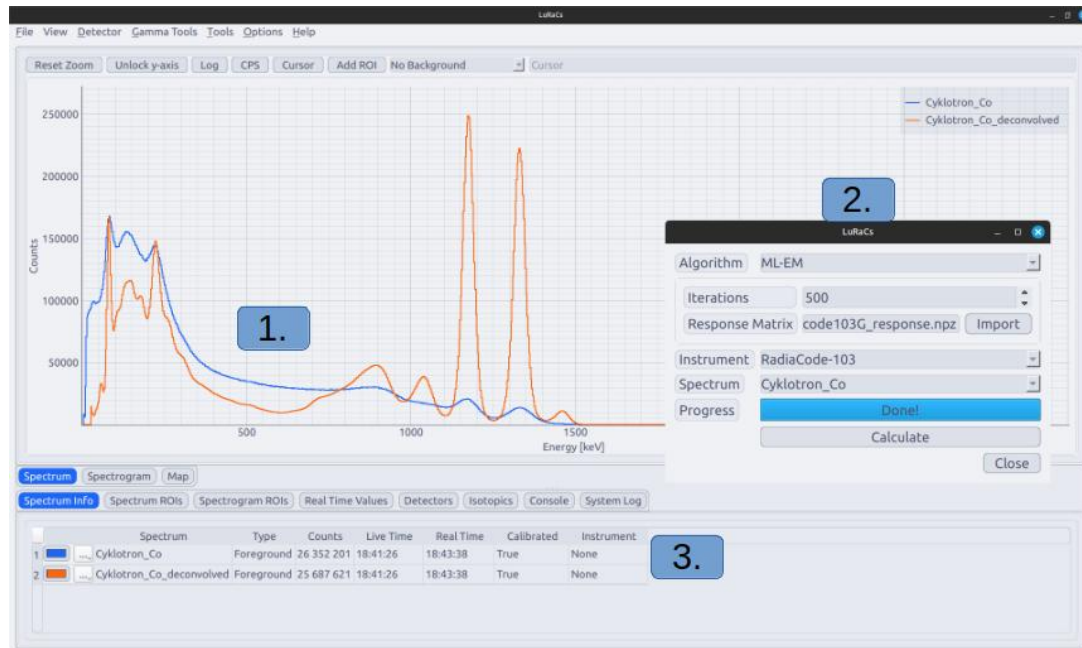


Figure 2.15: A Co-60 spectrum deconvolved using ML-EM and a response matrix for a Radiacode-103G detecting a parallel beam of monoenergetic photons.

1. The original Co-60 spectrum in blue and the deconvolved spectrum in orange.
2. Control window for the deconvolution operation, see Table 2.7.
3. The deconvolved spectrum is added as new spectrum, copying information from the original.

Table 2.7: Description of the deconvolution window.

Field	Description
Algorithm	Which algorithm should be used. Options are ML-EM (use this) and Generalized Richardson-Lucy .
Iterations	The number of iterations the algorithm should run.
(ML-EM) Response Matrix	The path to a .npz file containing the detector response matrix to be used.
Instrument	Instrument providing information the algorithm. For instance, Generalized Richardson-Lucy is based on the resolution saved in the instrument.
Spectrum	Which spectrum should be the operation be executed on.

2.3.3 Efficiency

The intrinsic efficiency is an important detector property and describes the fraction of photons hitting the detector that is actually detected.

2.3.4 Resolution

The energy resolution describes how well the detector can resolve detected photons of different energies. A low resolution results in broad peaks which can cover smaller peaks, making them harder to detect.

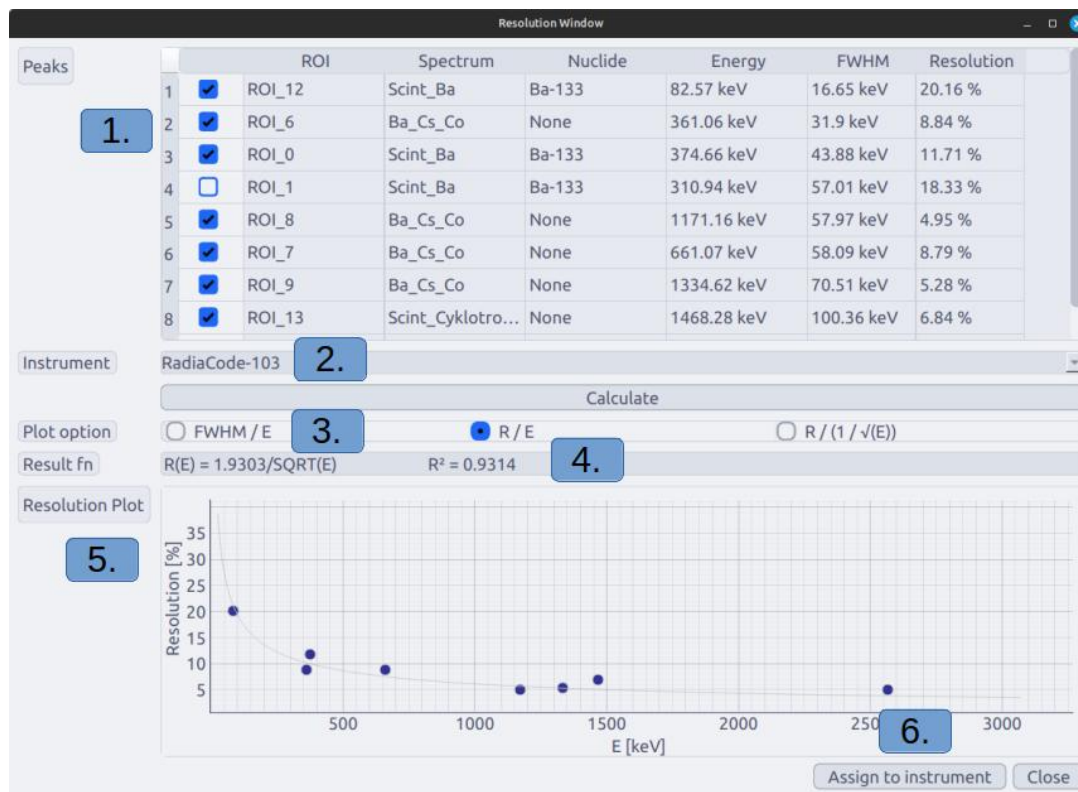


Figure 2.16: Resolution window with several different spectra measured with a 3"x3" NaI(Tl) scintillator loaded. Using **Tabbed Spectrum View** so the ROIs are only evaluating one spectrum each.

1. Fitted peaks from all loaded spectra and ROIs. A ROI without a fit can not be included. Unchecked entries will be ignored in the calculation.
2. Instrument to be given the calculation result. Can be Unique or Generic.
3. How the calculation results should be plotted. These do not affect the results, only the view.
4. The resolution function, coefficients are in arbitrary units and not % (% = au * 100). The R^2 value of the fitted function compared with the data points.
5. Assign the calculation results to the instrument set in (2.).

2.4 Command Line Arguments

Command Line Arguments can be provided when starting the application from a terminal.

Table 2.8: Command-line options available when starting LuRaCs.

Option	Description
General	
--debug	Launch the application in debug mode. A mock device is added automatically for testing and development.

Continued on next page

Table 2.8 – continued from previous page

Option	Description
<code>--headless</code>	Run the application without the graphical user interface, using terminal mode instead.
<code>--clear_local_cache</code>	Remove all locally stored files and exit the application.
Data Import	
<code>--import_spectrum <file></code> [file ...]	Load one or more spectrum files when starting the application. Each specified file must exist and be supported by the application's file importer.
<code>--import_rois <file></code>	Load regions of interest from the specified XML file.
<code>-roi <lower> <upper></code>	Add a region of interest with the specified lower and upper energy bounds in keV. The option can be specified multiple times to add several ROIs.
<code>--nuclides <nuclide></code> [nuclide ...]	Preset one or more nuclides in the isotopics interface. This option is applied when the graphical user interface is available.
Device Connections	
<code>-ble <device> [device ...]</code>	Attempt to establish Bluetooth Low Energy connections using the specified device names.
<code>--bluetooth <device></code> [device ...]	Long-form alias for <code>-ble</code> . Attempts Bluetooth Low Energy connections using the specified device names.
<code>-usb <device> [device ...]</code>	Attempt to establish USB connections using the specified device names. Matching is performed against detected USB device product names.
<code>-nw <ip> [ip ...]</code>	Attempt to connect to network devices using their IP addresses.
<code>--network <ip> [ip ...]</code>	Long-form alias for <code>-nw</code> . Connects to network devices using their IP addresses.
<code>--serial_gps <interface></code>	Connect a GPS receiver using a USB serial interface. The specified value identifies the serial interface used by the GPS receiver.
Maps	
<code>--map_url <URL></code>	Load an online map from the specified URL at application startup.
<code>--map_file <path></code>	Load a local offline map from the specified <code>.pmtiles</code> file at application startup.
<code>--map_url</code> and <code>--map_file</code>	Only one map source may be specified at startup. If both options are provided, no map is loaded and a warning is written to the application log.

2.5 Command Line Interface (CLI)

Interaction with LuRaCs is done primarily through the GUI but it also provides a terminal based command line interface, allowing for detector control without a GUI. This can be useful for remote measurements or measurements run on a Raspberry Pi attached to a drone. This mode shares most of the same commands as the GUI Console with the exception of visualisation tools.

This mode is started by running LuRaCs in a terminal with the `--headless` flag.

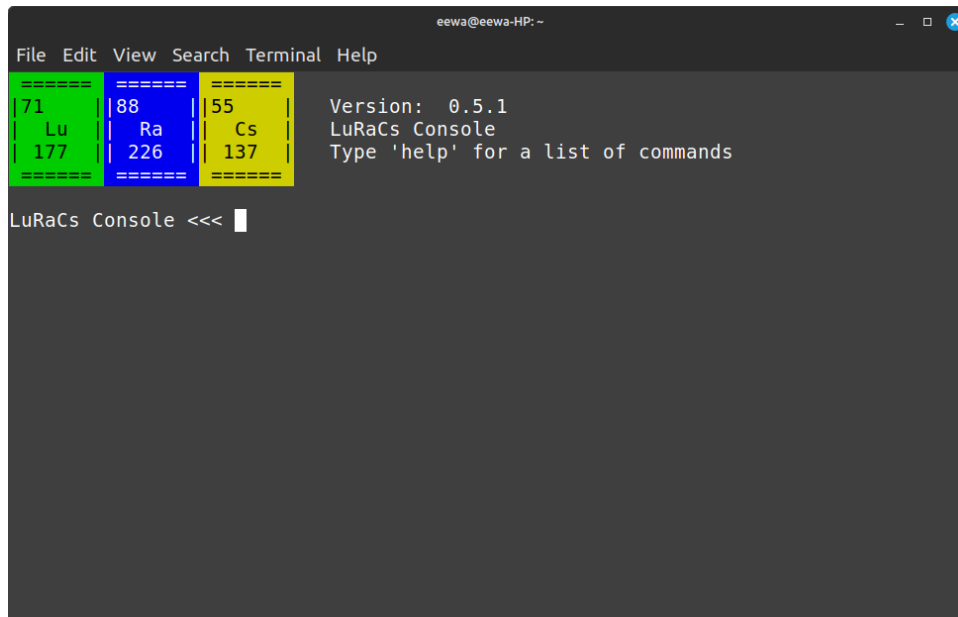


Figure 2.17: LuRaCs command line interface.

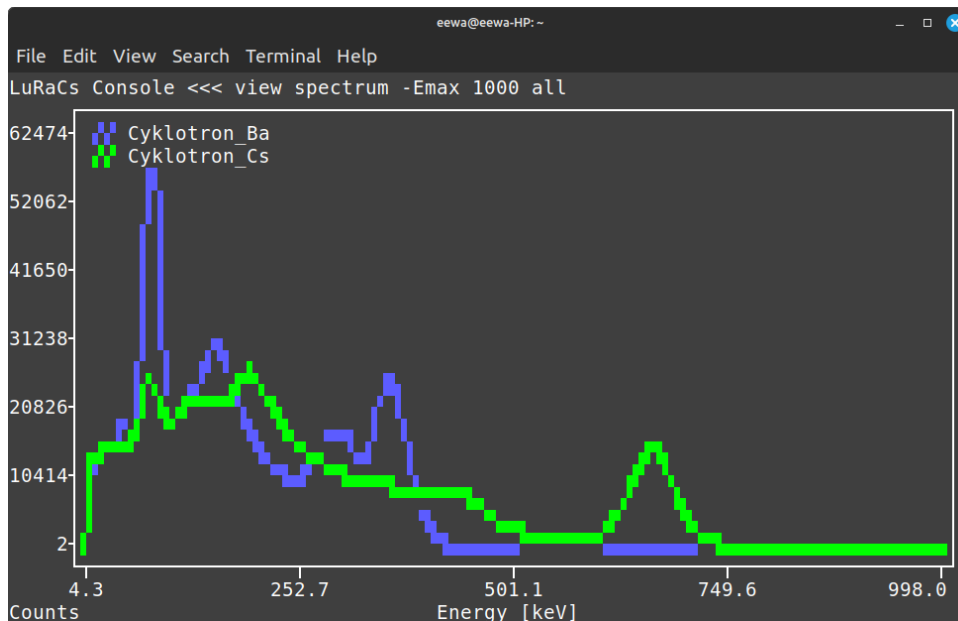
Figure 2.18: Two spectra viewed through the command line using the command `view spectrum -Emax 1000 all`. The `-Emax 1000` flag limits the x-axis to < 1000 keV.

Table 2.9: Command-line interface command reference.

Command	Description
General	
<code>help</code>	Display the available commands and their descriptions.
<code>clear</code>	Clear the console and display the application logo.
<code>exit, quit, shutdown</code>	Exit the application.
Device Management	
<code>device scan usb</code>	Scan for connected USB devices.

Continued on next page

Table 2.9 – continued from previous page

Command	Description
device scan ble	Scan for nearby Bluetooth Low Energy devices.
device connect usb <device>	Connect to a USB device matching the specified name.
device connect ble <device1> [device2 ...]	Connect to one or more Bluetooth Low Energy devices.
device disconnect <device>	Disconnect the specified device.
device disconnect all	Disconnect all connected devices.
Resource Listing	
list devices	Display all connected devices.
list spectra	Display all loaded spectra.
list spectrogram	Display all loaded spectrograms.
list rois	Display all loaded regions of interest.
Indexes	
index spectrum	Display indexed spectrum files.
index spectrogram	Display indexed spectrogram files.
index roi	Display indexed ROI files.
Viewing Data	
view spectrum <name>	Display the specified spectrum. The spectrum can be selected by name or by its index in the loaded spectrum list.
view spectrum -n <name>	Specify the spectrum to display by name.
view spectrum all	Display all loaded spectra on the same plot.
view spectrum -A	Display all loaded spectra on the same plot.
view spectrum -Emin <energy>	Set the lower energy limit of the displayed spectrum, in keV.
view spectrum -Emax <energy>	Set the upper energy limit of the displayed spectrum, in keV.
view spectrum -cps	Display the spectrum as counts per second (CPS) instead of total counts.
view spectrum -log	Display the spectrum using the logarithmic representation.
view spectrum -bkgSub	Display the background-subtracted spectrum.
view rois	Display the currently loaded regions of interest.
view log	Display recent application log entries.
ROI Management	
roi <lower> <upper>	Add a region of interest between the specified lower and upper bounds.
roi -c	Clear all regions of interest.
Spectrogram Management	
spectrogram start <device>	Start a spectrogram for the specified device.
spectrogram start all	Start spectrograms for all connected devices.
spectrogram pause <name>	Pause the specified spectrogram.
spectrogram pause all	Pause all running spectrograms.
spectrogram unpause <name>	Resume the specified spectrogram.
spectrogram unpause all	Resume all paused spectrograms.
spectrogram load <path>	Load a spectrogram from the specified file.
spectrogram unload <name>	Unload the specified spectrogram.

Continued on next page

Table 2.9 – continued from previous page

Command	Description
spectrogram unload all	Unload all loaded spectrograms.
-n <name>	Specify a custom name when starting a spectrogram.
-i <seconds>	Set the spectrogram save interval. Default: 1 second.
-c <count>	Set the number of spectra to concatenate before saving. Default: 1.
Map Management	
map url <https-URL>	Load an online map from the specified HTTPS URL.
map file <path>	Load a local offline map from a PMTiles file.
Monitoring	
watch	Continuously display live information from connected devices. Press Ctrl+C to stop.
Automation	
exec <script-file>	Execute commands from a script file. Empty lines and lines beginning with # are ignored.

2.6 Detector Connection & Control

Connecting detectors using the GUI is facilitated through a dialog for each type of connection, the are found under **Detector -> Connect Bluetooth/USB/Network**. Connecting to bluetooth and USB requires the application the scan for available devices, the serial number of the found devices are then displayed for selection.

Connecting the a network devices requires knowing the device IP and type beforehand. Connecting to network detectors have the unique option to use the detector as a GPS. This is primarily intended for the Ortec Detective X which has a built in GPS.

When connecting to serial GPS devices the application scans serial ports and lists suggestions but the port can also be entered manually. The baud rate is typically 4800 but a selection of other common rates are also presented.

2.7 Data Store

The *Data Store* allows for saving data objects in the application it self for easy access. It also allows for limited editing of previously saved objects. It contains the following object types *Spectra*, *Spectrograms*, *ROIs*, *Unique Instruments* and *Generic Instruments*.

All data items saved in the Data Store can be exported to a single zip-file with **File -> Export Data Store**. Such a file can be imported with **File -> Import Data Store**.

Spectrum Library

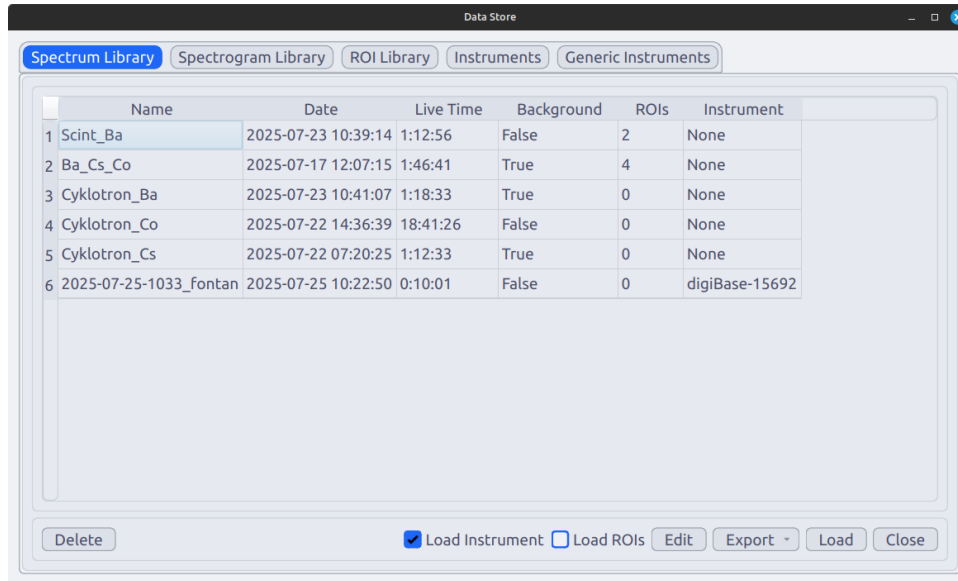


Figure 2.19

Table 2.10: Description of fields in toolbar at the bottom of the Spectrum Library-tab

Field	Description
Delete	Remove selected items.
Load Instrument	Load the instrument contained in the specific spectrum file. If not selected LuRaCs will attempt to match the instrument name to an existing <i>Unique Instrument</i> save in the Data Store.
Load ROIs	Should the ROIs contained in the spectrum be loaded with the spectrum.
Edit	Edit the spectrum name or remark.
Export	Export the spectrum. Available formats: XML, CSV and xlsx .
Load	Load the spectrum into view.
Close	Close the data store.

Table 2.11: Description of columns in the Spectrum Library-tab

Column	Description
Name	Spectrum name
Date	Date when the measurement of the foreground started
Live Time	Live time of the foreground
ROIs	How many ROIs are saved with the spectrum, if any.
Instrument	Which instrument is saved with the spectrum if any. If an instrument with the same name exists the in the Data Store, it will be used instead of the instrument saved with the spectrum. This can be changed in setting.

Spectrogram Library

All created spectrograms are placed in the Spectrogram Library. From here they can be loaded back for analysis or continued measurement.

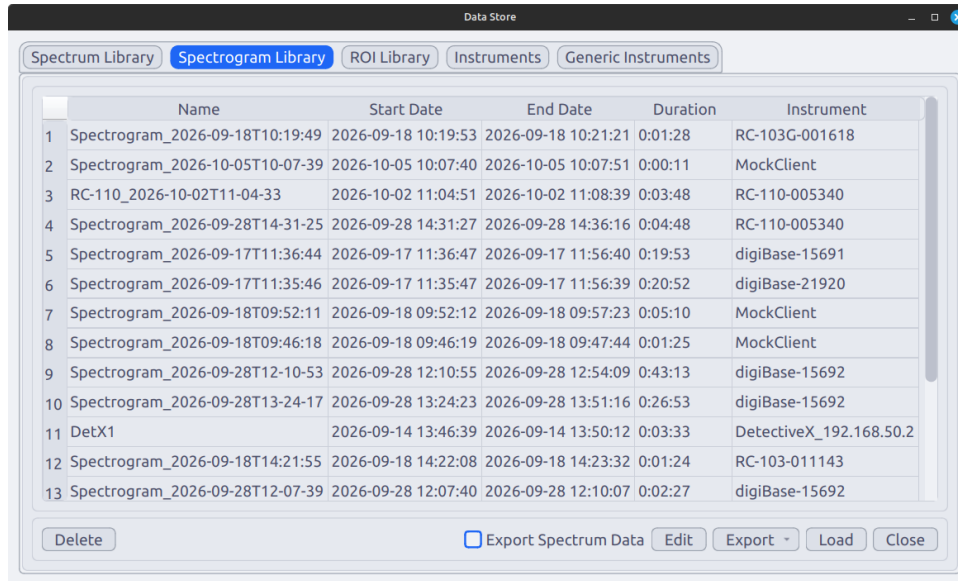


Figure 2.20

Table 2.12: Description of fields in toolbar at the bottom of the Spectrum Library-tab

Field	Description
Delete	Remove selected items.
Export Spectrum Data	If checked the data for individual channels are included when exporting, everything else is exported the same.
Edit	Edit the spectrogram name.
Export	Export the spectrogram to another file. Available formats: .db (SQLite3) and xlsx .
Load	Load the spectrum into view.
Close	Close the data store.

Table 2.13: Description of columns in the Spectrum Library-tab

Column	Description
Name	Spectrogram name
Start Date	Date when the measurement started.
End Date	Date when the measurement was last stopped.
Duration	Total measurement duration.
Instrument	Identifier for the detector used the measure the spectrogram.

ROI Library

The ROI Library allows for saving references to ROIs to then be loaded to a spectrum or spectrogram. This means the roi options like the energy bounds, fitting options and assigned nuclide are saved but data like fitting parameters or peak counts are not.

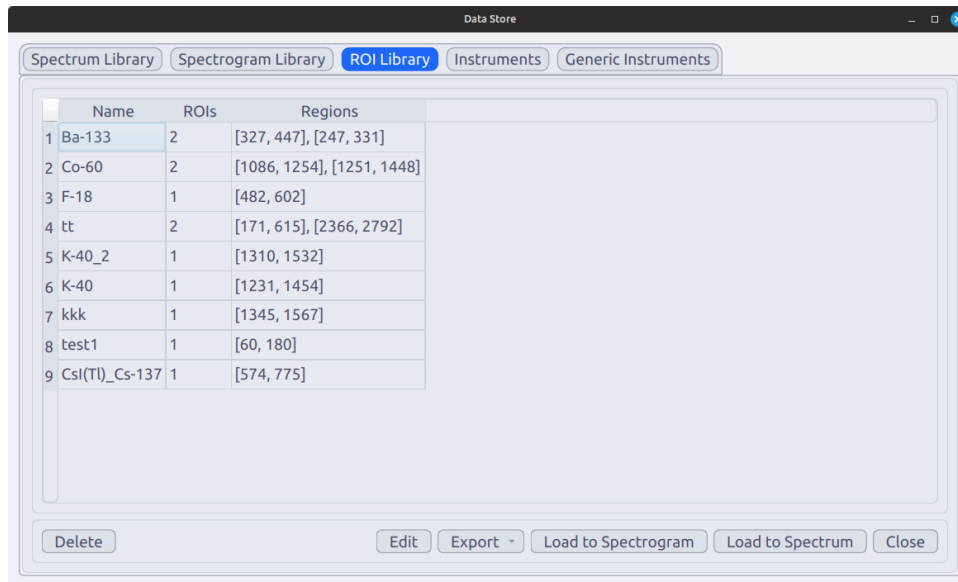


Figure 2.21

Table 2.14: Description of fields in toolbar at the bottom of the Spectrum Library-tab

Field	Description
Delete	Remove selected items.
Edit	Edit the ROI collection name and the ROIs contained within, see Figure 2.22.
Export	Export the ROIs to another file. Available formats: .xml.
Load to Spectrogram	Load the ROIs as spectrogram ROIs.
Load to Spectrum	Load the ROIs as spectrum ROIs.
Close	Close the data store.

Table 2.15: Description of columns in the Spectrum Library-tab

Column	Description
Name	Spectrogram name
Start Date	Date when the measurement started.
End Date	Date when the measurement was last stopped.
Duration	Total measurement duration.
Instrument	Identifier for the detector used the measure the spectrogram.

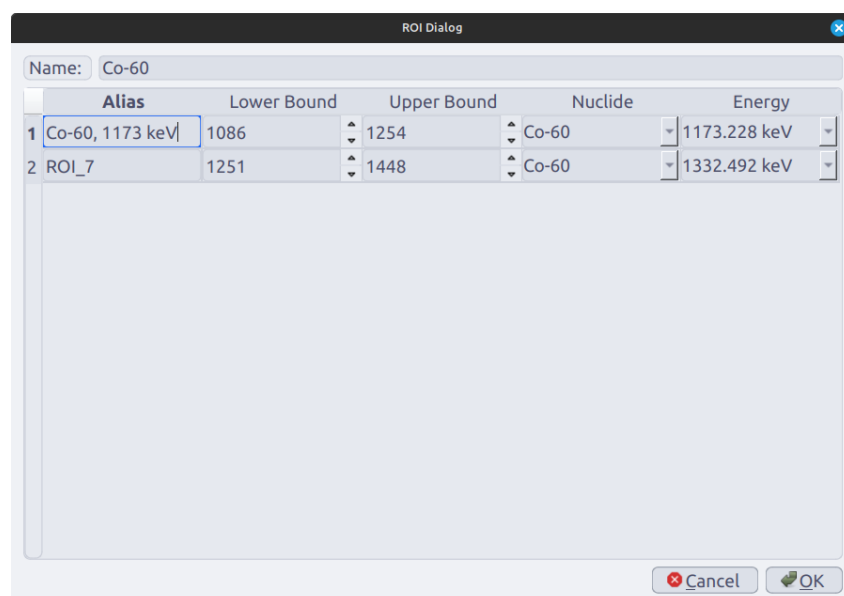


Figure 2.22: ROI editor inside the Data Store. ROI alias/name can be changed, upper and lower bounds and nuclide and emission can be selected from the nuclide library.

Unique-/Generic Instruments

The tabs for the Unique and Generic Instruments are very similar and treated in the same section.

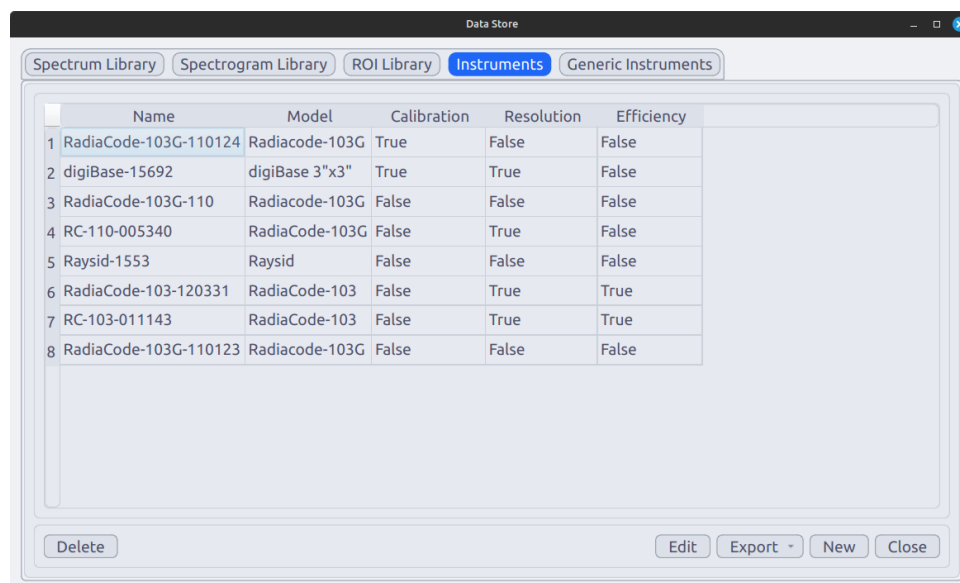


Figure 2.23: Instruments-tab in the Data Store containing Unique Instruments.

Table 2.16: Description of fields in toolbar at the bottom of the Spectrum Library-tab

Field	Description
Delete	Remove selected items.
Edit	Edit an existing instrument, see Figure 2.24.
Export	Export the instrument data to another file. Available formats: .xml.
New	Create a new instrument. <i>Unique Instruments</i> can inherit data from <i>Generic Instruments</i> .
Close	Close the data store.

Table 2.17: Description of columns in the Spectrum Library-tab

Column	Description
Name	Instrument name (Only for <i>Unique Instrument</i>)
Model	Name of the instrument model.
Calibration	If the instrument stores an energy calibration. (Only for <i>Unique Instrument</i>)
Resolution	If the instrument stores an energy resolution.
Efficiency	If the instrument stores an intrinsic efficiency.

The screenshot shows the 'Instrument Dialog' window with the following fields and plots:

- Generic Instruments:** None
- Instrument Name:** digiBase-15692
- Instrument Model:** digiBase 3"x3"
- Manufacturer:** Ortec
- Detector Material:** NaI(Tl)
- Remarks:** Use +850V bias
- Shape:** Cuboid
- Dimensions:** Height: 7,62 cm, Width: 7,62 cm, Length: 0,00 cm
- Uncertainties:** Height: ± 0,00 cm, Width: ± 0,00 cm, Length: ± 0,00 cm
- Resolution:** [2026-09-27 23:03] fn = 1.893/SQRT(E)
- Efficiency:** [None] fn = None, params = None

Two plots are displayed:

- Resolution Plot:** Y-axis is Resolution [%] (0 to 40), X-axis is Energy [keV] (0 to 3000). Data points show a decreasing trend from ~35% at 100 keV to ~5% at 2500 keV.
- Efficiency Plot:** Y-axis is Int. Eff. [%] (0 to 1.0), X-axis is Energy [keV] (0 to 0.9). The plot is currently empty.

Buttons at the bottom right: Cancel, OK.

Figure 2.24: The dialog for creating a new instrument or editing an existing one.

2.8 Settings

The settings are divided into **Settings** which contain basic and cosmetic settings, and **Advanced Settings** containing settings that users rarely need to interact with.

Table 2.18: Application settings.

Setting	Description
Appearance	
Theme	Select the visual theme used by the application interface and plots.
Plot Colour Scheme	Select the colour scheme used when automatically assigning colours to plotted spectra.
Spectrum Draw Style	Select how spectra are rendered. Available options are <i>Pen</i> , <i>Brush</i> , and <i>Pen + Brush</i> .
Font Size	Set the application font size in points.
Spectrum Plot xMax	Set the maximum energy displayed on spectrum plots, in keV.
Verbose Calculation Logging	Enable additional logging during calculations. This can be useful when diagnosing calculation or processing problems.
Import Options	
Load ROIs on import	Automatically load regions of interest when importing a spectrum from a file.
Load instrument on import	Automatically load instrument information when importing a spectrum from a file.
Load calibration from instrument	Load the energy calibration associated with the instrument when importing data from the Data Store or from a file.

Table 2.19: Advanced application settings.

Setting	Description
Device Running	
Update Loop Delay	Set the delay between iterations of the device monitoring and update loop, in seconds.
Spectrum Update Delay	Set the delay between spectrum updates during live acquisition, in seconds.
Device Scanning	
GUI Scan Length	Set the duration of Bluetooth Low Energy scans initiated through the graphical user interface, in seconds.
Headless Scan Length	Set the duration of Bluetooth Low Energy scans when no the scan is not tracked in the GUI, in seconds.
Optimizer	
Optimizer Max Iter	Set the maximum number of iterations allowed by the optimizer.
Optimizer Tolerance	Set the convergence tolerance used by the optimizer.
Use Chi ² Weight	Enable χ^2 weighting during optimization.
Logging	
Log Exceptions	Enable logging of unhandled exceptions.
Write logs to file	Enable writing log messages to the application log file.
Write logs to console	Enable writing log messages to the terminal or console.
Headless Log Buffer Length	Set the maximum number of log entries retained in the in-memory log buffer. Changes to this setting require an application restart.

Continued on next page

Table 2.19 – continued from previous page

Setting	Description
Spectrogram	
Spectrogram View Buffer	Set the number of recent spectrogram data points retained for display.
Maps	
Save extra GPS data	Enable saving additional GPS information with map-related data.

3 Examples

3.1 Quick Starting Several Radiacode Detectors for mobile measurements

This examples demonstrates how the terminal can be used to effectively control several detectors at once, without having to repeat the connection and start measurement procedure for each one. In this example three Radiacode detectors are used but the principle is the same for all other types of detector. The code bellow illustrates the commands start an offline map, connect the GPS and instruments and start spectrograms for all detectors.

```
# Start the map first, cacheing a large map with require all available resources
↪ and will cause the application the freeze momentarily. This can disturb the
↪ sampling of the detectors.
map file ~/Downloads/sverige.pmtiles

# Once the map is loaded connect the GPS
# This assumes only one GPS device is connected to the PC, if several are
↪ connected the first serial port is chosen
gps connect

# This command will scan all bluetooth devices and match the given name to the
↪ serial number of the found devices
device connect ble Radia

# This will connect all detectors with 'radia' in the name, matching is case
↪ insensitive
# Check if all detectors connected, if not repeat the above command
list devices

# Start recording spectrograms for all devices
spectrogram start all

# When the measurements are concluded stop the recordings
spectrogram pause all
```

3.2 Connecting Detective X using direct ethernet

LuRaCs communicates with a Detective X detector using the *Mobile Field Kit* API described in the Detective X manual. Using this interface requires a network connection between the instrument and a host computer. A simple and stable way to establish such a connection is a direct ethernet cable from the instrument to the PC with no router in between. This method requires some setup before *LuRaCs* can connect to a Detective X and is described in this example.

The first step is to configure the ethernet interface on the Detective X. Go to **Settings** → **Communications** → **Ethernet**. Ensure ethernet is enabled and disable DHCP. Manually enter the IP-address and network mask. A good choice of IP is **192.168.50.2**, as long as does not interfere with another connected device, and set the network mask to **255.255.255.0**.

The second step is to configure the PC to have a static IP address on the ethernet connection. The used IP will be **192.168.50.1** and the network mask 255.255.255.0, leave gateway empty. Examples are shown for Windows 11 and Linux Mint but the principle is the same for most systems.

Once the network is configured check the connection by pinging Detective X. On most Linux systems, if the ping fails try restarting the network manager with the following command `sudo systemctl restart NetworkManager`. If that does not work, try rebooting the PC.

If the ping succeeds everything is ready. In *LuRaCs* go to **Devices → Connect Network** and enter the instrument IP, 192.168.50.2 in this example.

3.3 Preparing and Running the LU mobile detector setup

This setup consists of two 4 L NaI(Tl) scintillators with digiBase MCAs and one HPGe detector with a digiDart system mounted in a car, equipped with an external usb GPS device.

1. Ensure that the digiBase firmware is imported. **File → Drivers**
2. Connect the detectors, neither the digiBase nor the digiDart save their calibration when used with LuRaCs. **Detector → Connect USB**
3. Enable the correct voltage on each detector, this might be remembered by the detector. **Detector-tab → 'A detector Row' → '...' -button → Settings → MCA-tab → HV: Enable**
4. (Optional) Calibrate the detectors with the K-40 and Tl-208 peaks and the 0 channel using the Calibration tool. Apply to the spectrum. **Gamma Tools → Calibration**
5. With the calibration a new instrument can now be created from the Detectors-tab to save the calibration. It will automatically be fetched next time the detector is connected. **Detectors-tab → 'A detector Row' → '...' -button → Make Instrument → 'Save Calibration' is checked**
6. Start the map. If an offline map is used start it before any spectrogram, loading a large only map might interfere with sampling of the instruments, messing up the even save interval. **Map-tab → Map Options → Load Offline Map**
7. Connect the GPS. **Detector → Connect Serial GPS**, determine the correct port if required and connect. 4800 is likely the correct Baud Rate.
8. Check that the GPS has a signal. **Map-tab → Line Bellow the map → 'Fix' should be 'Yes'**
9. Ensure all detectors are running by clearing all spectra. **Detector → Clear All Spectra**
10. Start the spectrograms individually, **Spectrum-tab → Start New** or use the console, **Console → 'spectrogram start all'**.

4 Internal Function

4.1 Code Structure

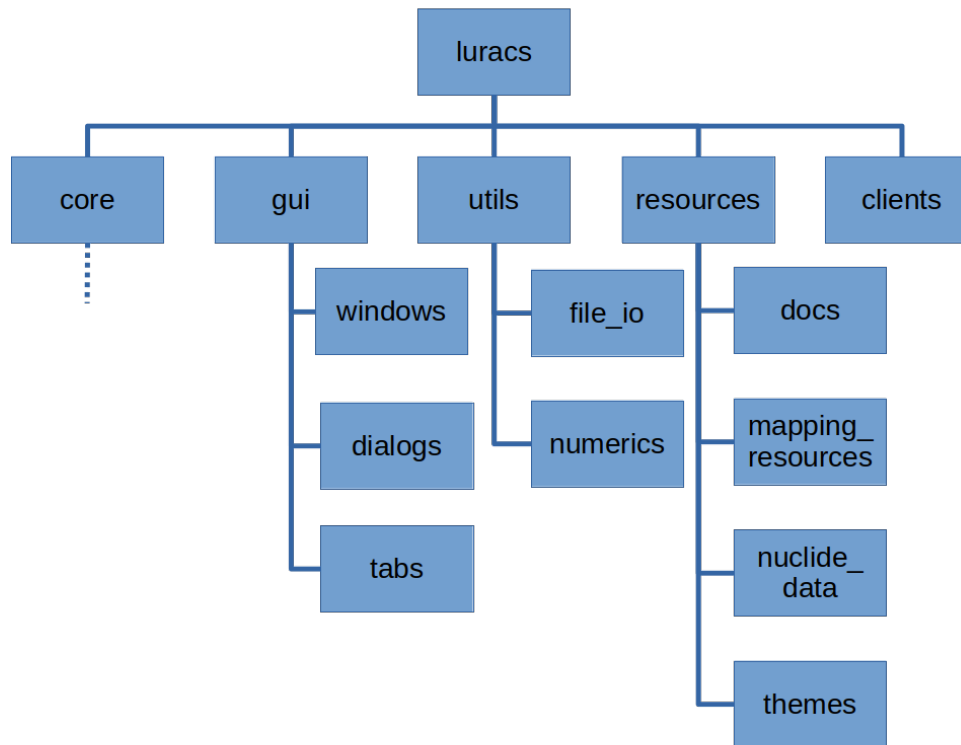


Figure 4.1: Flowchart illustrating the structure of the LuRaCs code

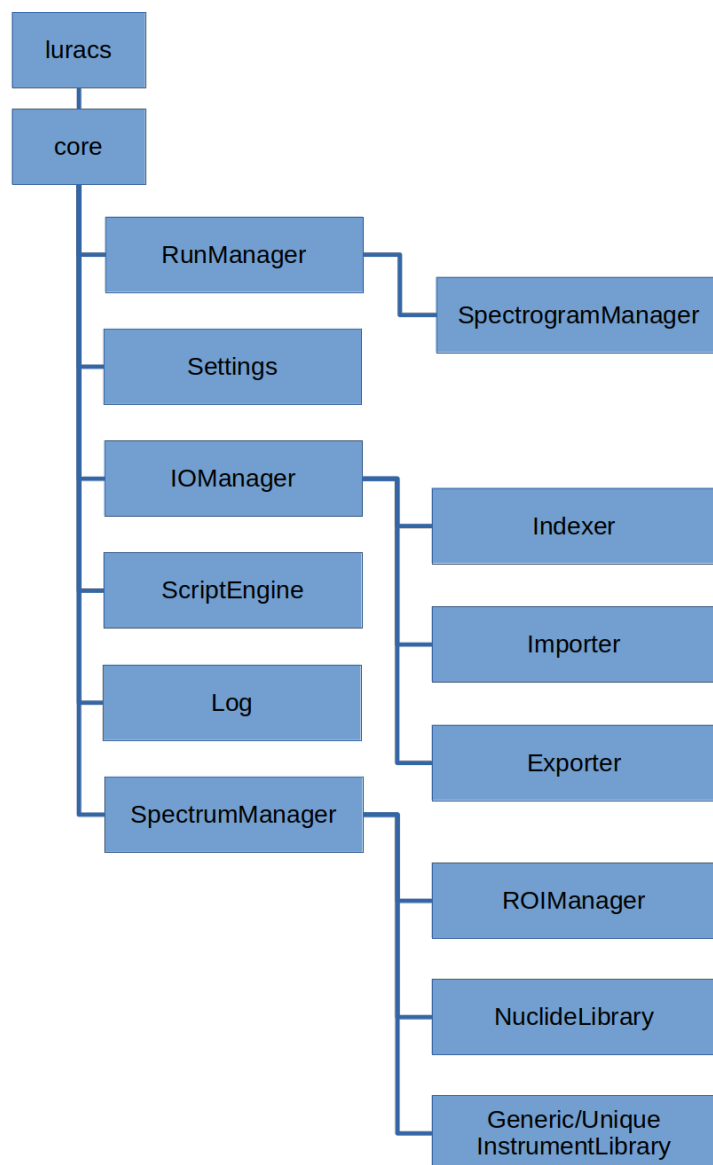
core

Figure 4.2: Flowchart of the structure of the LuRaCs core module

The core of the application is built on a number of singleton classes that manage the global state of the program. These are reachable from anywhere in the program.

Settings The settings contain modifiable parameters that affect the running of the program. It is divided in the the following parts. Settings can also be used for communication within the program as when a setting is changed using the `update_setting` function a signal is emitted from **Settings** with the group of the setting, name of the setting and the new value.

- **Appearance** contain simple settings mostly controlling appearance of the GUI and preferences for data import.
- **Paths** contain file and system paths to resources and stored data.
- **Temp** contain values that are reset upon shutdown.

- **State** saves mostly last used values for convenience and is capable of saving and loading complex data types like dequeues.
- **Advanced** contains advanced options that most users rarely need to change.

RunManager The RunManager handles connecting and sampling instruments. It owns its own thread running an async event loop, this to avoid instrument sampling being affected by events in the GUI thread and vice versa. Threadsafe communications are handled using Qt signals. The RunManager also owns the **SpectrogramManager** which tracks spectrogram and manages spectrogram ROIs.

ScriptEngine The ScriptEngine is responsible for converting text commands into actions. Like the RunManager it owns its own thread and communications are made using Qt signals. Available commands are described in the section on the CLI.

Log The Log exports a reference to the python logger "Application".

IOManager The IOManager provides functions for import and export of various files and manages indexes of internally stored file for the Data Store.

SpectrumManager The SpectrumManager is one of the most critical parts of the application. It owns all spectra, both from files and devices. Modifications of spectra are handled through it. It also owns the following components.

- **NuclideLibrary** contains loaded nuclide data and some functions for interacting with it.
- **ROIManager** handles spectrum ROIs and updates peak fits.
- **GenericInstrumentLibrary** contains generic instruments based on instrument models that can be used to store characteristic data and be used as basis for Unique-Instruments.
- **UniqueInstrumentLibrary** contains unique instruments that can be assigned to spectra. They differ from GenericInstruments in that they store information unique to each device like energy calibration.

The core also contains the helper class **Calculator** that is designed to easily assign a CPU intensive task to a separate thread and signal back the result using a Qt signals.

4.2 Wrapping Detectors

The implementation of detectors is written to be extensible and make it easy to implement new instruments. This is facilitated through inheritance from an abstract class **DeviceWrapper**. This wrapping hides the individual device implementations and LuRaCs only interact through this wrapper interface. Creating a subclass from **DeviceWrapper** automatically registers the new wrapper for use as a detector connection.

4.3 Algorithms

4.3.1 Peak Fitting

Peak fitting are based in ROIs in which a peak is assumed to be. Each ROI can contain one peak but ROIs can merge so several peaks can share the same background estimation. One a ROI is defined the fitting process is as follows.

1. Determine a number of channels starting at the ROI and counting inwards on which to base the background.
2. A polynomial is fitted using the edge channels and their corresponding counts.
3. The polynomial is evaluated for every channel in the ROI or the merged ROI region and the result subtracted from the respective channels.
4. One or the sum of several Gauassian functions are fitted to the remainder.
5. Peak characteristics are extracted from the fitted paramters.

The uncertainties of the peak fitting is calculated as follows. Firstly, the counts in channel i , N_i is assumed to follow Poisson distribution, thus $\sigma_i = \sqrt{N_i}$. The uncertainty of each channel is used as weights for the polynomial fit.

4.3.2 Peak Search

4.3.3 Deconvolution

The deconvolution method implemented is based on the *Maximum Likelihood - Expectation Maximisation* or ML-EM algorithm.

Before reconstruction the response matrices are processed and rows for the requested values are interpolated. The method for interpolation is described by Jandel et al.².

4.4 File Types

4.4.1 Spectrum, ROI and Instrument (.xml)

Spectra are stored in the XML format and is interoperable with the ANSI/IEEE N42.42 format³.

ROIs and Instrument data are stored as an extension to the N42 standard using the *LRC* namespace. All datatypes are converted to strings before XML serialisation. The tables bellow describe the XML fields and their corresponding Python type.

²M Jandel et al. "Decomposition of Continuum y-Ray Spectra Using Synthesized Response Matrix". In: *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment* 516.1 (Jan. 1, 2004), pp. 172–183. ISSN: 0168-9002. DOI: 10.1016/j.nima.2003.07.047.

³"Documentary Standards: ANSI/IEEE N42.42 Standard". In: *NIST* (). URL: <https://www.nist.gov/programs-projects/documentary-standards-ansiieee-n4242-standard> (visited on 10/06/2026).

Table 4.1: Fields stored in the XML representation of spectrum data.

XML field	Python type	Description
RadMeasurement/@id	str	Identifier of the radiation measurement.
MeasurementClassCode	str	Specifies whether the measurement is a foreground or background spectrum.
StartDateTime	str	Measurement start time formatted as an ISO 8601 date-time string.
RealTimeDuration	str	Real measurement duration encoded as an ISO 8601 duration.
Spectrum/ @energyCalibrationReference	str	Reference to the energy calibration used by the spectrum.
Spectrum/@id	str	Identifier of the spectrum.
LiveTimeDuration	str	Measurement live time encoded as an ISO 8601 duration.
ChannelData	np.ndarray	Spectrum channel counts, converted to integer values before serialization.

Table 4.2: Fields stored in the XML representation of a ROI.

XML field	Python type	Description
Roi/@id	str	Internal tag identifying the ROI.
Roi/@alias	str	User-defined name or alias of the ROI.
Roi/@spectrum_ref	str	Name of the spectrum associated with the ROI.
Roi/@version	str	Version of the XML representation.
PeakContinuum/@merge	bool	Indicates whether the ROI is merged with other ROIs.
PeakContinuum/@movable	bool	Indicates whether the ROI can be moved interactively.
PeakContinuum/ @background_subtracted	bool	Indicates whether background subtraction has been applied.
PeakContinuum/ RegionCounts	float	Total counts within the ROI.
PeakContinuum/LiveTime	float	Measurement live time.
PeakContinuum/ LowerEnergy	float	Lower energy boundary of the ROI.
PeakContinuum/ UpperEnergy	float	Upper energy boundary of the ROI.
Fit	str	Type of peak fitting method applied to the ROI.
Fit/@chi2_weighted_err	bool	Indicates whether weighted χ^2 errors are used.
BackgroundType	str	Type of background model used for fitting.
Peak/ BackgroundParameters	np.ndarray	Parameters of the fitted background model.
Peak/FitParameters	np.ndarray	Optimized parameters of the peak-fitting function.
Peak/FitParametersErr	np.ndarray	Uncertainties of the fitted peak parameters.
Peak/Centroid	list	Fitted centroid and its uncertainty.
Peak/StandardDeviation	list	Fitted standard deviation and its uncertainty.
Peak/Amplitude	list	Fitted amplitude and its uncertainty.
Peak/PeakArea	list	Integrated peak area and its uncertainty.
Peak/NetCounts	list	Net peak counts and their uncertainty.
Peak/BkgCounts	list	Estimated background counts and their uncertainty.
Peak/GrossCounts	list	Gross peak counts and their uncertainty.
Nuclide/Name	str	Name of the associated nuclide.
Nuclide/Energy	list	Emission energy and its uncertainty in keV.
Nuclide/Intensity	list	Emission intensity and its uncertainty in percent.
Nuclide/Type	str	Type of nuclear emission. <i>g</i> or <i>x-ray</i> .
Nuclide/EmissionOrigin	str	Origin of the nuclear emission information.

Table 4.3: Basic instrument information stored in XML.

XML field	Python type	Description
Instrument/@generic_instrument	bool	Indicates whether the instrument is generic rather than unique.
Instrument/Name	str	Name of the instrument.
Instrument/Model	str	Model of the instrument.
Instrument/Manufacturer	str	Manufacturer of the instrument.
Instrument/ DetectorMaterial	str	Material of the detector.
Instrument/DetectorShape	str	Shape of the detector.
Instrument/DetectorType	str	Type of detector.
Instrument/ DetectorDimensions	tuple	Physical dimensions of the detector in centimetres.
Instrument/ DetectorDimensionsUncertainty	tuple	Uncertainties of the detector dimensions in centimetres.
Instrument/Remark	str	Optional free-text remark associated with the instrument.

Table 4.4: Energy resolution information stored in XML.

XML field	Python type	Description
Resolution/@type	str	Type of resolution model, set to energy .
Resolution/CreatedTS	datetime	Date and time at which the resolution model was created.
Resolution/Function	str	Function used to describe the energy resolution.
Resolution/Parameters	list	Parameters of the energy resolution function.
Resolution/ ResolutionPoints/@energy_unit	str	Unit of energy for the resolution points, specified as keV.
Resolution/ ResolutionPoints/@FWHM_unit	str	Unit of FWHM, specified as keV.
Resolution/ ResolutionPoints/ DataPoint/FWHM	float	Measured full width at half maximum.
Resolution/ ResolutionPoints/ DataPoint/FWHMUncertainty	float	Uncertainty of the measured FWHM.
Resolution/ ResolutionPoints/ DataPoint/Energy	float	Energy corresponding to the measured resolution.

Table 4.5: Intrinsic efficiency information stored in XML.

XML field	Python type	Description
Efficiency/@type	str	Type of efficiency model, set to <code>intrinsic</code> .
Efficiency/CreatedTS	datetime	Date and time at which the efficiency model was created.
Efficiency/Function	str	Function used to describe the intrinsic efficiency.
Efficiency/Parameters	list	Parameters of the intrinsic efficiency function.
Efficiency/ ParametersUncertainty	list	Uncertainties of the intrinsic efficiency parameters.
Efficiency/ EfficiencyPoints/@energy_ unit	str	Unit of energy for the efficiency points, specified as keV.
Efficiency/ EfficiencyPoints/ @efficiency_unit	str	Unit of efficiency, specified as unitless.
Efficiency/ EfficiencyPoints/ DataPoint/Efficiency	float	Measured intrinsic efficiency.
Efficiency/ EfficiencyPoints/ DataPoint/ EfficiencyUncertainty	float	Uncertainty of the measured efficiency.
Efficiency/ EfficiencyPoints/ DataPoint/Energy	float	Energy corresponding to the measured efficiency.
Efficiency/Description	str	Description of the intrinsic efficiency data or model.

Table 4.6: Energy calibration information stored in XML.

XML field	Python type	Description
Calibration/ PolynomialOrder	int	Order of the polynomial used for energy calibration.
Calibration/Coefficients	list	Polynomial coefficients used for channel-to-energy conversion.
Calibration/ CalibrationPoints/ @energy_unit	str	Unit of calibration energy, specified as keV.
Calibration/ CalibrationPoints/ DataPoint/Energy	float	Known energy of a calibration point.
Calibration/ CalibrationPoints/ DataPoint/Channel	float	Channel corresponding to the calibration energy.
Calibration/CreatedTS	datetime	Date and time at which the energy calibration was created.

4.4.2 Spectrogram (.db)

Spectrograms are saved to an SQLite3 database file. One file contains one spectrogram. Saved files can be loaded and measurement resumed. A spectrogram database contains three tables described in Table 4.7, 4.8 and 4.9.

Table 4.7: Schema of the `header` table.

Column	Type	Description
id	INTEGER	Identifier of the header record.
created	INTEGER	Creation time as a Unix timestamp.
device_id	TEXT	Identifier of the device producing the data.
channels	INTEGER	Number of spectrum channels.
calibration	TEXT	Calibration coefficients as a json list.
concat	INTEGER	Indicates whether the channels have been concatenated by a factor of 1 (not concatenated), 2, 4 or 8.
save_interval	REAL	Interval between data saves.
meta	TEXT	Additional metadata as json.

Table 4.8: Schema of the `summary` table.

Column	Type	Description
id	INTEGER	Identifier of the summary record.
accumulated_spectrum	BLOB	Accumulated spectrum data, zlib compressed array.
total_duration	REAL	Total duration of the recording in [s].
total_dose	REAL	Total accumulated dose in [μ Sv].
last_update	INTEGER	Timestamp of the last update as a Unix timestamp.

Table 4.9: Schema of the `spectrogram` table.

Column	Type	Description
<code>id</code>	INTEGER	Identifier of the spectrogram record.
<code>timestamp</code>	REAL	Timestamp of the measurement as a Unix timestamp.
<code>avg_cps</code>	INTEGER	Average counts per second multiplied by 1000.
<code>avg_dr</code>	INTEGER	Average dose rate as [$\mu\text{Sv/s} \times 1000$].
<code>temperature</code>	INTEGER	Temperature during the measurement in [$^{\circ}\text{C} \times 1000$].
<code>latitude</code>	REAL	Latitude of the measurement location.
<code>longitude</code>	REAL	Longitude of the measurement location.
<code>spectrum</code>	BLOB	Recorded spectrum during interval, zlib compressed array.
<code>meta</code>	BLOB	Additional metadata as a json string encoded with utf-8 and zlib compressed.

4.4.3 Detector Response Matrix (.npz)

Detector response matrices are stored as serialized and compressed numpy arrays (.npz) with pickle disabled. Each files contains three arrays described in Table 4.10.

Table 4.10: Contents of a response matrix .npz file.

Key	dtype	shape	Description
<code>response_matrix</code>	float32	2D (m x n)	Detector response values. First index is the simulated photon energy and second index is the normalized count bins ($\# \text{bin} / \# \text{fired photons}$)
<code>indices</code>	float32	1D (m)	Energy of the simulated monoenergetic photons of the corresponding row in <code>response_matrix</code> in keV
<code>bin_centres</code>	float32	1D (n)	Energy of the bin centres of the detector energy response in keV.

4.4.4 Simple map tracks (.geojson)

Spectrograms are the primary way to store GPS position data these are however not directly compatible with any other software without custom scripts. A spectrogram can be converted to the *GeoJSON* format, with the loss of some information, for compatibility and easier sharing. The *GeoJSON* format is generally compatible with dedicated GIS software, like *QGis*, for viewing measurements outside the embedded map.

Table 4.11: Collection-level contents of the GeoJSON mapping data file.

Key	dtype	Description
type	string	GeoJSON object type. Always set to "FeatureCollection".
name	string	Name or title of the mapping dataset.
metadata	object	Metadata describing the mapping session, including device information, acquisition times, and additional metadata.
metadata.device_id	string	Identifier of the device used to acquire the measurements.
metadata.start	string	ISO 8601 timestamp indicating the start of the measurement period.
metadata.end	string or null	ISO 8601 timestamp indicating the end of the measurement period. Set to null if unavailable.
features	array	Collection of georeferenced measurement points.

Table 4.12: Contents of individual measurement features in the GeoJSON mapping data. Accessed as `features.xxx`

Key	dtype	Description
type	string	GeoJSON feature type. Always set to "Feature".
geometry	object	Geographic representation of the measurement. The geometry is a GeoJSON Point.
geometry.type	string	Geometry type. Always set to "Point".
geometry.coordinates	float	Longitude and latitude of the measurement location, given in GeoJSON order as [longitude, latitude].
properties.timestamp	string	ISO 8601 timestamp indicating when the measurement was recorded.
properties.count_rate	float	Measured count rate at the corresponding geographic location.
properties.dose_rate	float	Measured dose rate at the corresponding geographic location.
properties.other_data_points	on	Additional measurements points, typically produced by spectrogram ROIs, saved as a JSON-object.

4.5 Embedded map

The embedded map is rendered using *MapLibre GL JS*⁴ in a QWebEngine widget. The packages for MapLibre is included in the LuRaCs code and does not require any installation. MapLibre handles requesting tiles is connected to an online API it requires minimal

⁴*Maplibre/Maplibre-GL-Js*. MapLibre, Sept. 21, 2026. URL: <https://github.com/maplibre/maplibre-gl-js> (visited on 09/21/2026).

work. Using maps offline works by using an embedded tile server run using *uvicorn* and *fastapi* to supply MapLibre with tiles. The embedded tile server runs on host 127.0.0.1 and port 8080. PMTiles are loaded using a fork of the *pmtiles* Python package, having improved cacheing and performance, incorporated in the LuRaCs code.

The code is written so that the the Python side handles most of the logic, using only simple calls to JS for creating or updating points, etc. To minimise the overhead incurred from the *Python* $\leftrightarrow$ *JS*-bridge send lists as a one object and let JS iterate over it, as opposed to sending every point over the bridge.

References

- “Documentary Standards: ANSI/IEEE N42.42 Standard”. In: *NIST* (). URL: <https://www.nist.gov/programs-projects/documentary-standards-ansiieee-n4242-standard> (visited on 10/06/2026).
- Jandel, M et al. “Decomposition of Continuum γ -Ray Spectra Using Synthesized Response Matrix”. In: *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment* 516.1 (Jan. 1, 2004), pp. 172–183. ISSN: 0168-9002. DOI: 10.1016/j.nima.2003.07.047.
- Maplibre/Maplibre-Gl-Js*. MapLibre, Sept. 21, 2026. URL: <https://github.com/maplibre/maplibre-gl-js> (visited on 09/21/2026).
- Sandialabs/InterSpec*. Sandia National Laboratories, Aug. 28, 2025. URL: <https://github.com/sandialabs/InterSpec> (visited on 08/29/2025).